

Optimization Methods

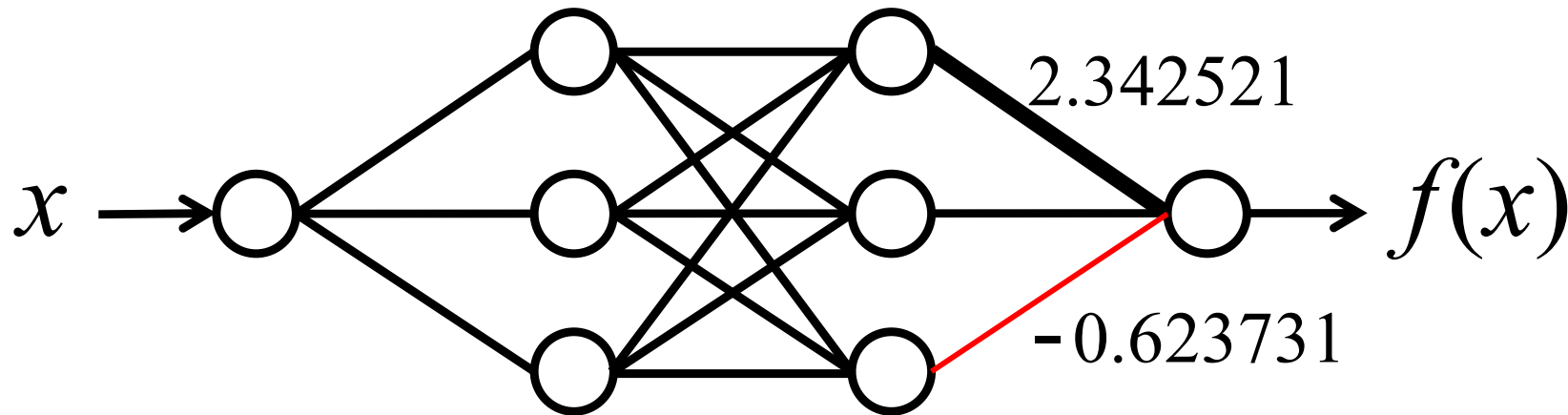
1. Introduction.
2. Greedy algorithms for combinatorial optimization.
3. LS and neighborhood structures for combinatorial optimization.
4. Variable neighborhood search, neighborhood descent, SA, TS.
5. Branch and bound algorithms, and subset selection algorithms.
6. Linear programming problem formulations and applications.
7. Linear programming algorithms.
8. Integer linear programming algorithms.
9. Unconstrained nonlinear optimization and gradient descent.
10. Newton's methods and Levenberg-Marquardt modification.
11. Quasi-Newton methods and conjugate direction methods.
12. Nonlinear optimization with equality constraints.
13. Nonlinear optimization with inequality constraints.
14. Problem formulation and concepts in multi-objective optimization.
15. Search for single final solution in multi-objective optimization.
- 16: Search for multiple solutions in multi-objective optimization.

Learning and Optimization

Simple Learning Problem

Training Data: $(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m)$

Testing Data: $(x_{m+1}, y_{m+1}), (x_{m+2}, y_{m+2}), \dots, (x_{m+n}, y_{m+n})$



Learning: To adjust the connection weights to minimize the following cost function (training data accuracy):

$$z(\mathbf{w}) = [(y_1 - f(x_1))^2 + \dots + (y_m - f(x_m))^2]/m$$

Evaluation: Testing data accuracy:

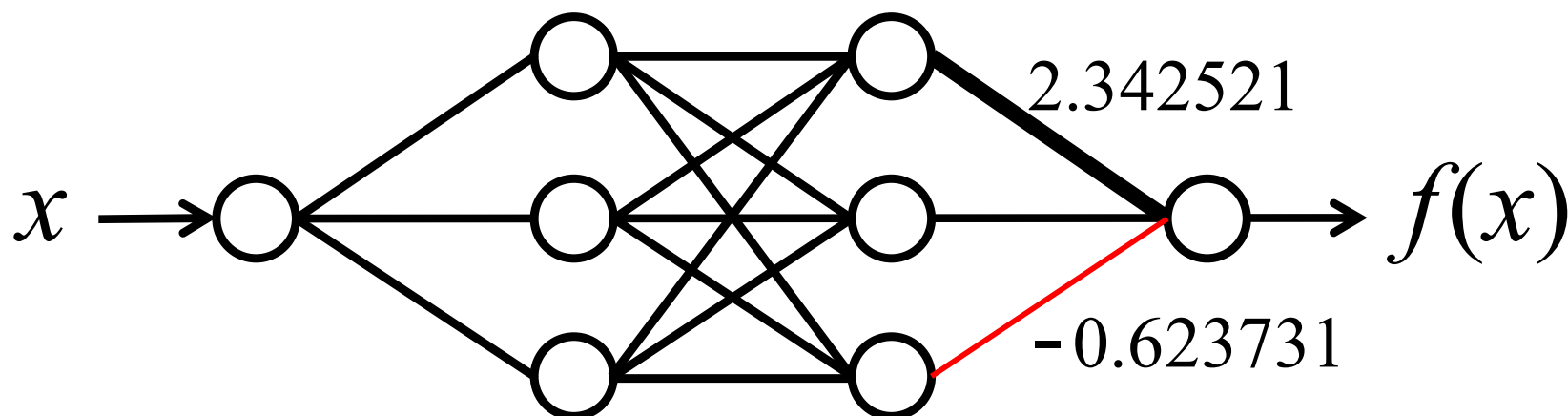
$$z(\mathbf{w}) = [(y_{m+1} - f(x_{m+1}))^2 + \dots + (y_{m+n} - f(x_{m+n}))^2]/n$$

Learning and Optimization

Optimization Problem

Training Data: $(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m)$

~~Testing Data: $(x_{m+1}, y_{m+1}), (x_{m+2}, y_{m+2}), \dots, (x_{m+n}, y_{m+n})$~~



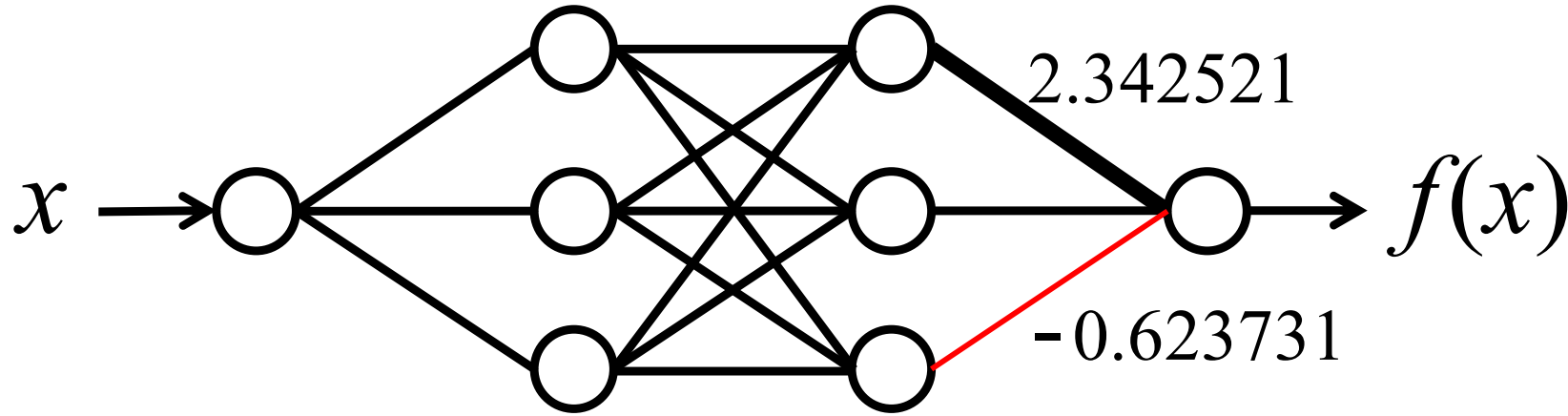
Optimization: To adjust the connection weights to minimize the following cost function (training data accuracy):

$$z(\mathbf{w}) = [(y_1 - f(x_1))^2 + \dots + (y_m - f(x_m))^2]/m$$

Evaluation: Training data accuracy:

$$z(\mathbf{w}) = [(y_1 - f(x_1))^2 + \dots + (y_m - f(x_m))^2]/m$$

Learning and Optimization



Learning: To adjust the connection weights to minimize the following cost function (training data accuracy):

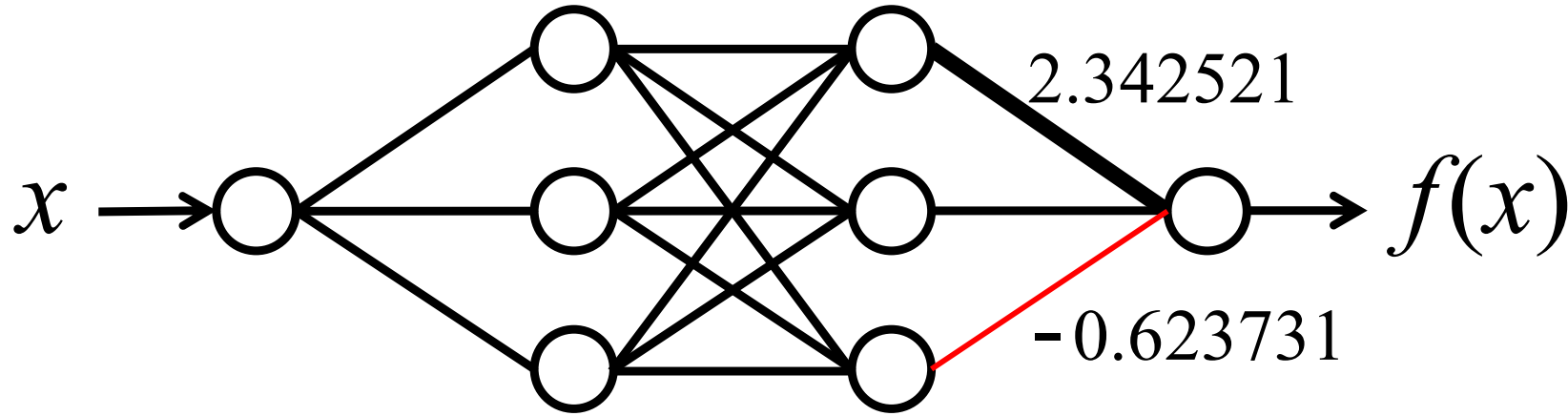
$$z(\mathbf{w}) = [(y_1 - f(x_1))^2 + \dots + (y_m - f(x_m))^2]/m$$

Optimization: To adjust the connection weights to minimize the following cost function (training data accuracy):

$$z(\mathbf{w}) = [(y_1 - f(x_1))^2 + \dots + (y_m - f(x_m))^2]/m$$

Basically, learning can be viewed as optimization (whereas generalization instead of optimization is emphasized in learning).

Learning and Optimization



Evaluation in learning: Testing data accuracy:

$$z(\mathbf{w}) = [(y_{m+1} - f(x_{m+1}))^2 + \dots + (y_{m+n} - f(x_{m+n}))^2]/n$$

Evaluation in optimization: Training data accuracy:

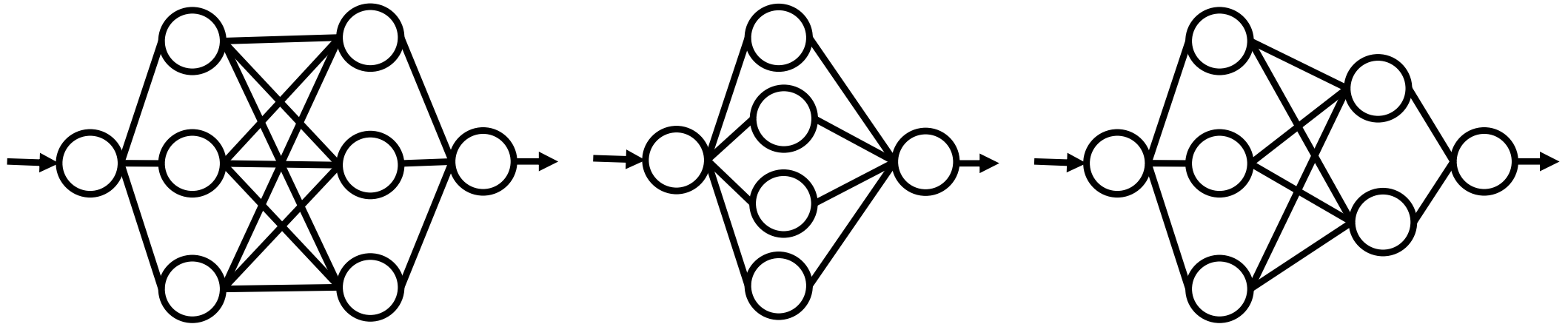
$$z(\mathbf{w}) = [(y_1 - f(x_1))^2 + \dots + (y_m - f(x_m))^2]/m$$

The final goals are different: Unseen testing data accuracy in learning, and given training data accuracy in optimization.

Neural Architecture Search

Training Data: $(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m)$

Testing Data: $(x_{m+1}, y_{m+1}), (x_{m+2}, y_{m+2}), \dots, (x_{m+n}, y_{m+n})$



Learning: To find an architecture and connection weights to minimize the following cost function:

$$z(\mathbf{w}) = [(y_1 - f(x_1))^2 + \dots + (y_m - f(x_m))^2]/m$$

Evaluation: Testing data accuracy:

$$z(\mathbf{w}) = [(y_{m+1} - f(x_{m+1}))^2 + \dots + (y_{m+n} - f(x_{m+n}))^2]/n$$

Question. Is this task learning or optimization ?

Categorization of Optimization Problems

Decision Variable-Based Categorization: Continuous or Discrete (Combinatorial)?

1. Continuous Optimization Problems

- Linear Programming Problems
- Nonlinear Programming Problems
- Learning of Neural Networks (Weight Adjustment)

2. Discrete (Combinatorial) Optimization Problems

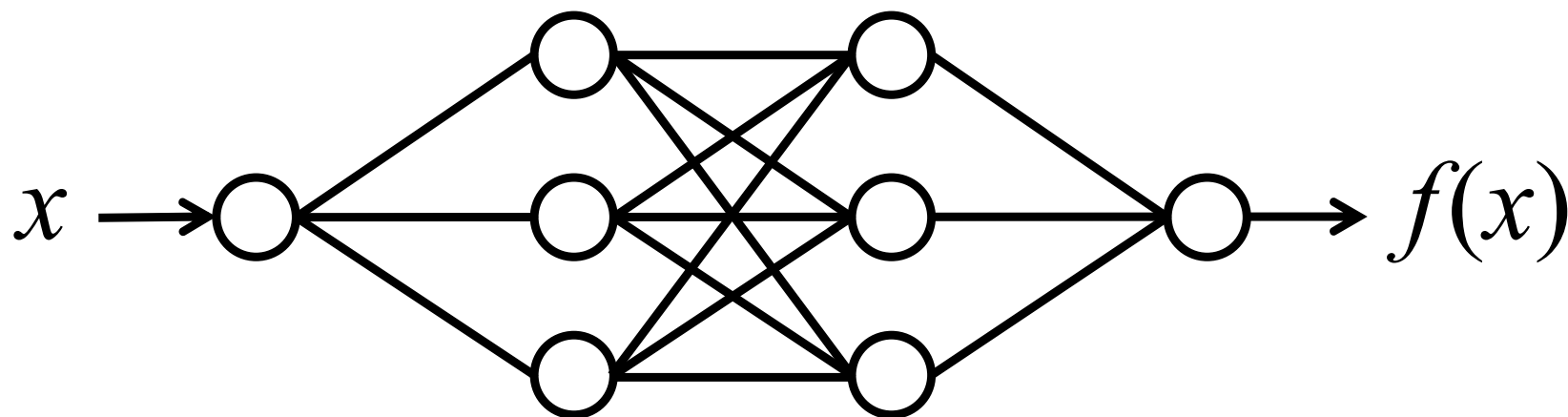
- Integer Programming Problems
- Scheduling Problems, Knapsack Problems, TSP
- Structure Learning of Neural Networks

3. Mixture of 1 and 2

- Mixed Integer Programming Problems
- Design Optimization (Length, Choice of Parts & Materials)
- Neural Network Design (Structure & Weights)

Continuous Optimization Problem

Training Data: $(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m)$

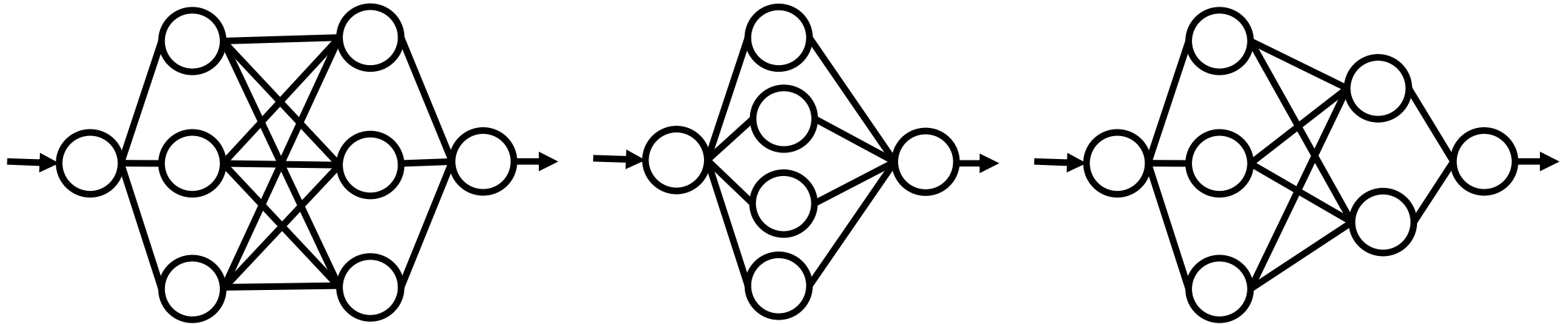


Optimization: To adjust the connection weights to minimize the following cost function (training data accuracy):

$$z(\mathbf{w}) = [(y_1 - f(x_1))^2 + \dots + (y_m - f(x_m))^2]/m$$

Mixed Optimization Problem (Continuous & Combinatorial)

Training Data: $(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m)$



Optimization: To find an architecture and connection weights to minimize the following cost function:

$$z(\mathbf{w}) = [(y_1 - f(x_1))^2 + \dots + (y_m - f(x_m))^2]/m$$

Categorization of Optimization Problems

Objective Function-Based Categorization 1:

Number of Objectives: Single or Multiple?

1. Single-Objective Optimization Problems

Many textbook optimization problems (test problems) have only a single objective such as the profit maximization in a knapsack problem and the processing time minimization in a scheduling problem. In machine learning, the accuracy maximization can be viewed as a single-objective optimization problem.

2. Multi-Objective Optimization Problems

Almost all real-world problems have multiple objectives (e.g., cost minimization and performance maximization). In the structure learning of neural networks, the complexity minimization and the accuracy maximization can be viewed as a two-objective problem. However, in machine learning, they are combined as a weighted sum in order to handle them in the single-objective framework.

Categorization of Optimization Problems

Objective Function-Based Categorization 2: Objective Value: Exact Value or Others?

1. Standard Optimization Problems

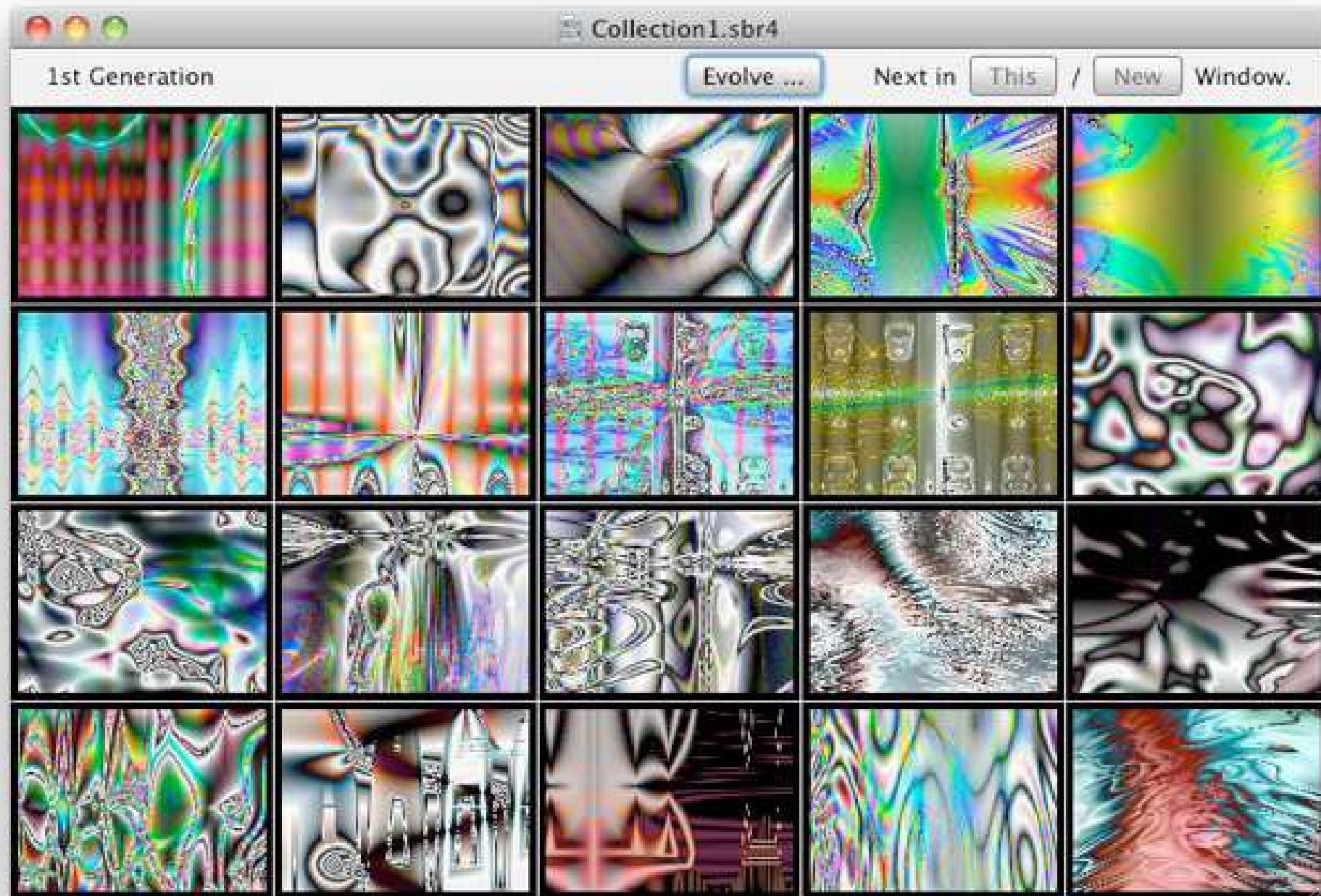
Objective values are exact values (e.g., real numbers).

2. Optimization Problems with Some Uncertainty

- Interval Programming Problems (e.g., [100\$, 110\$]).
- Stochastic Programming Problems (e.g., $N(100, 1^2)$).
- Fuzzy Programming Problems (e.g., about 100\$).
- Noisy Optimization Problems
(e.g., for the same solution, its objective value is 97, 102, 98, ...).
- Simulation-based Optimization Problems (e.g., multi-agent)
(Simulation results depend on randomness and parameters).
- Interactive Optimization Problems (e.g., CG design)
(Solutions are evaluated by human: noisy, inconsistent, tired).

Interactive Evolution of Computer Graphics

<http://www.intlab.soka.ac.jp/~unemi/sbart/4/ja/>



Categorization of Optimization Problems

Constraint Condition-Based Categorization: With or Without Constraint Conditions?

1. Unconstrained Optimization Problems

Optimization problems with no constraint conditions.

- Simple test problems such as “minimization of $(x-1)^2$ ”.
- TSP (Travelling Salesman Problem)
- Some Scheduling Problems

2. Constrained Optimization Problems

Optimization problems with constraint conditions.

- Linear Programming Problems
- Knapsack Problems
- Almost All Real-World Problems

Categorization of Optimization Algorithms

Is the obtained solution always optimal?

1: Exact Optimization Algorithms

- Linear Programming
- Dynamic Programming
- Branch-and-Bound Method

2: Approximation Algorithms

- Greedy Algorithms
- Local Search
- Genetic Algorithms
- Almost All Nonlinear Optimization Algorithms
- Learning of Connection Weights in Neural Networks

Discussions about Computation Load

For combinatorial optimization problems:

1: Exact Optimization Algorithms

Try to find the optimal solution without examining all possible combinations. However, in the worst case, the order of computation load is similar to the calculation of all possible combinations.

Example 1: Choice of a group of 5 students from 10 students. This is a simple problem. You can find the optimal solution by examining all combinations: 252 combinations ($10 \times 9 \times 8 \times 7 \times 6 / (5 \times 4 \times 3 \times 2 \times 1)$). If you can examine **one combination in one minute**, you will be able to examine all combinations and choose the best one **within 5 hours**.

Example 2: Choice of a group of 5 students from 100 students. This is basically the same problem as Example 1. The number of possible combinations is 75,287,520 ($100 \times 99 \times 98 \times 97 \times 96 / (5 \times 4 \times 3 \times 2 \times 1)$). If your computer can examine **1000 combinations in one second**, it can examine all combinations **within a day**.

Discussions about Computation Load

For combinatorial optimization problems:

1: Exact Optimization Algorithms

Example 3: Choice of a group of 10 students from 1000 students. This is basically the same problem as Examples 1-2. The number of possible combinations is . Even if your computer can examine **1 million combinations in one second**, it needs **about** **years** to examine all combinations.

Calculate now!

2: Simple Heuristic Algorithms

Example 3: Choice of a group of 10 students from 1000 students. First we evaluate each student independently. Then we choose the best 10 students. If we can use your one-million/second computer in Example 3, this calculation is within a second.

Difficulty: No consideration about interaction among students. The group of the best 10 students is not always the best team. However, this choice is usually much better than random choice.

Discussions about Computation Load

For combinatorial optimization problems:

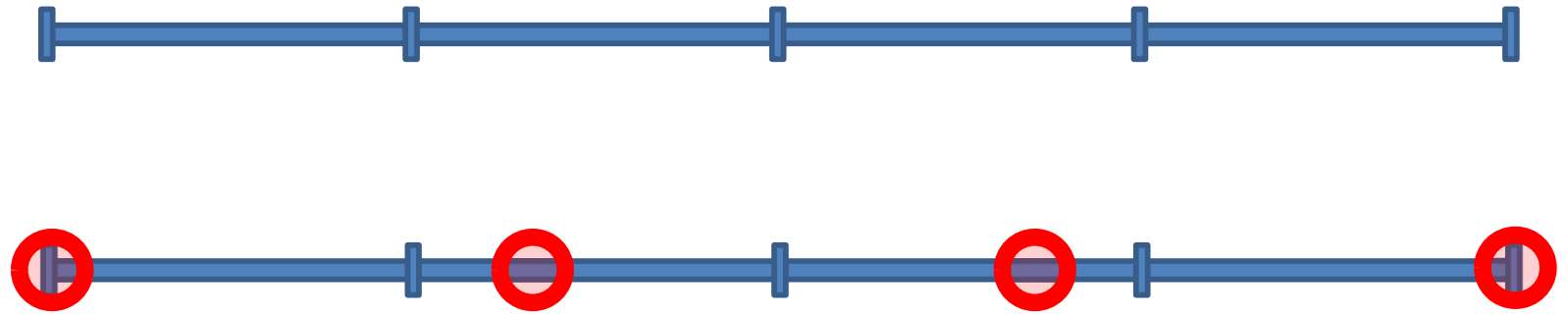
3: Greedy Algorithms

Example 3: Choice of a group of 10 students from 1000 students. First we evaluate each student independently, and choose the best student. Next we evaluate all combinations (999 combinations) of the selected student and another student, and choose the best combination. Then we evaluate all combinations (998 combinations) of the selected two students and another student, and choose the best combination. In this manner, we can choose a good combination of 10 students. If we can use your one-million/second computer in Example 3, this calculation is within a second.

Advantages: Maybe better than the choice of the best 10 students, still very fast. **Disadvantages:** The selection result can be totally different from the optimal combination since the selection is based on only a small number of possible combinations.

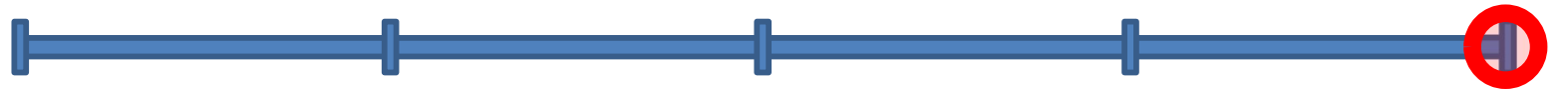
Simple Example: To select four points on the line to maximize the minimum distance between two points.

**Optimal
Solution**

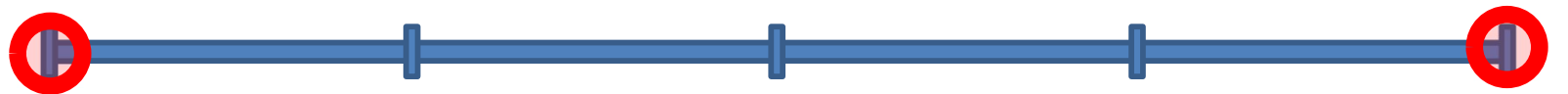


Greedy

1st Step



2nd Step



3rd Step



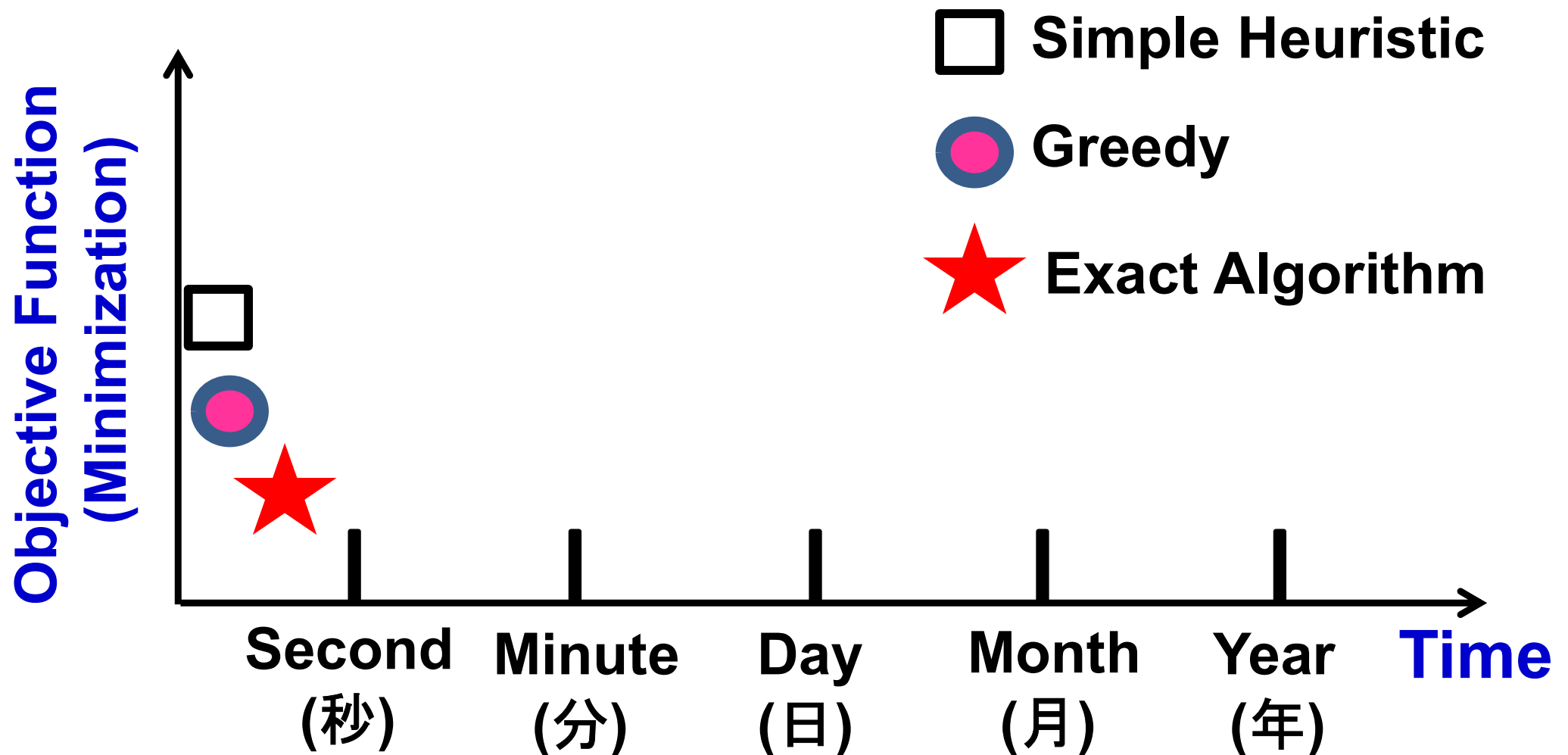
4th Step



Discussions about Computation Load

For combinatorial optimization problems:

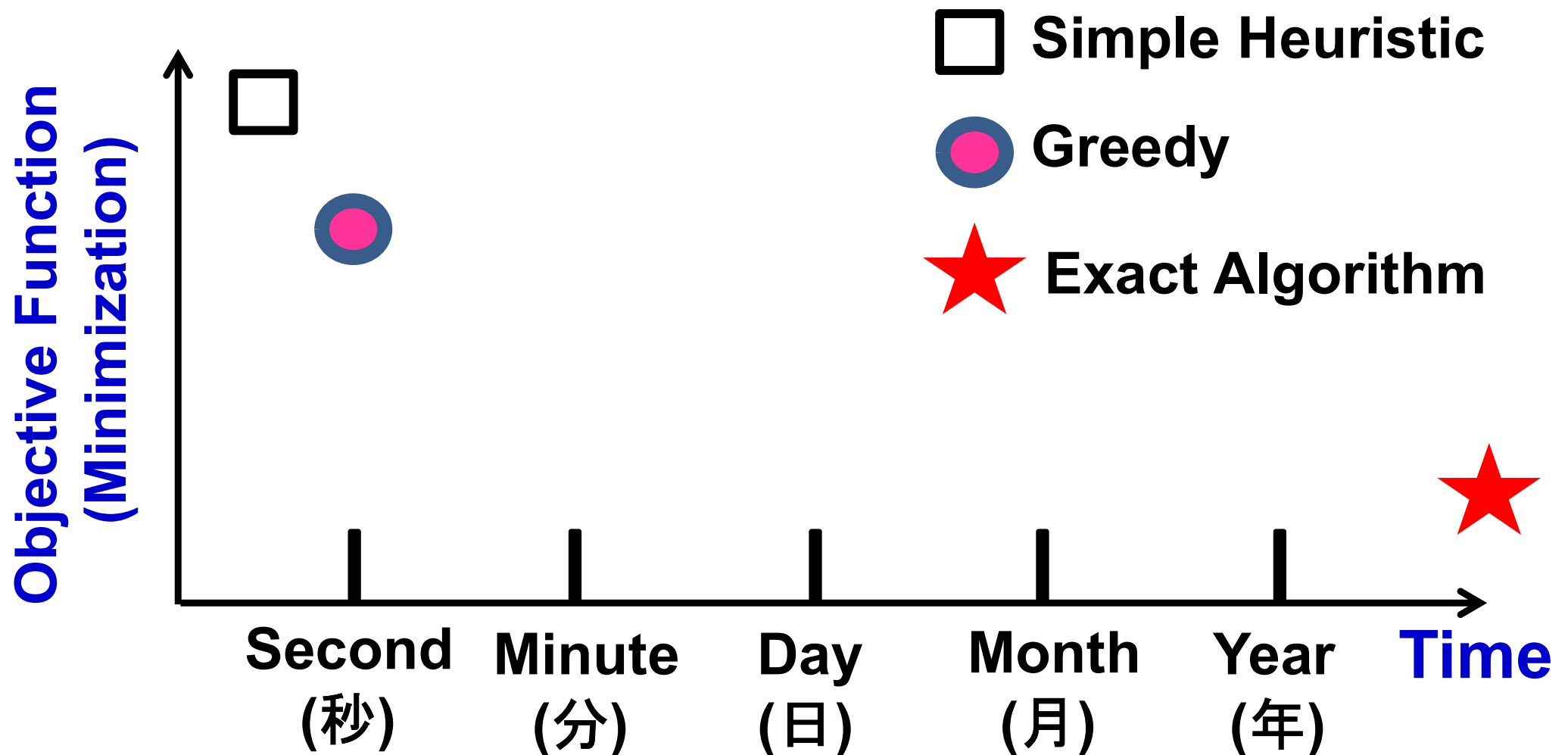
For Small Problem (Selection of 5 from 10)



Discussions about Computation Load

For combinatorial optimization problems:

For Large Problem (selection of 10 from 1000)



Discussions about Computation Load

For combinatorial optimization problems:

1: Exact Optimization Algorithms

Example 4: Assignment of 4 research topics to 4 students. The total number of different combinations is only $4 \times 3 \times 2 \times 1 = 24$.

Example 5: Assignment of 20 research topics to 20 students. The total number of different combinations is $20 \times 19 \times \dots \times 1 = 2.4 \times 10^{18}$. Even if we can use your **one-million/second** computer, we need **77,000 years** to examine all combinations.

2: Simple Heuristic Algorithms

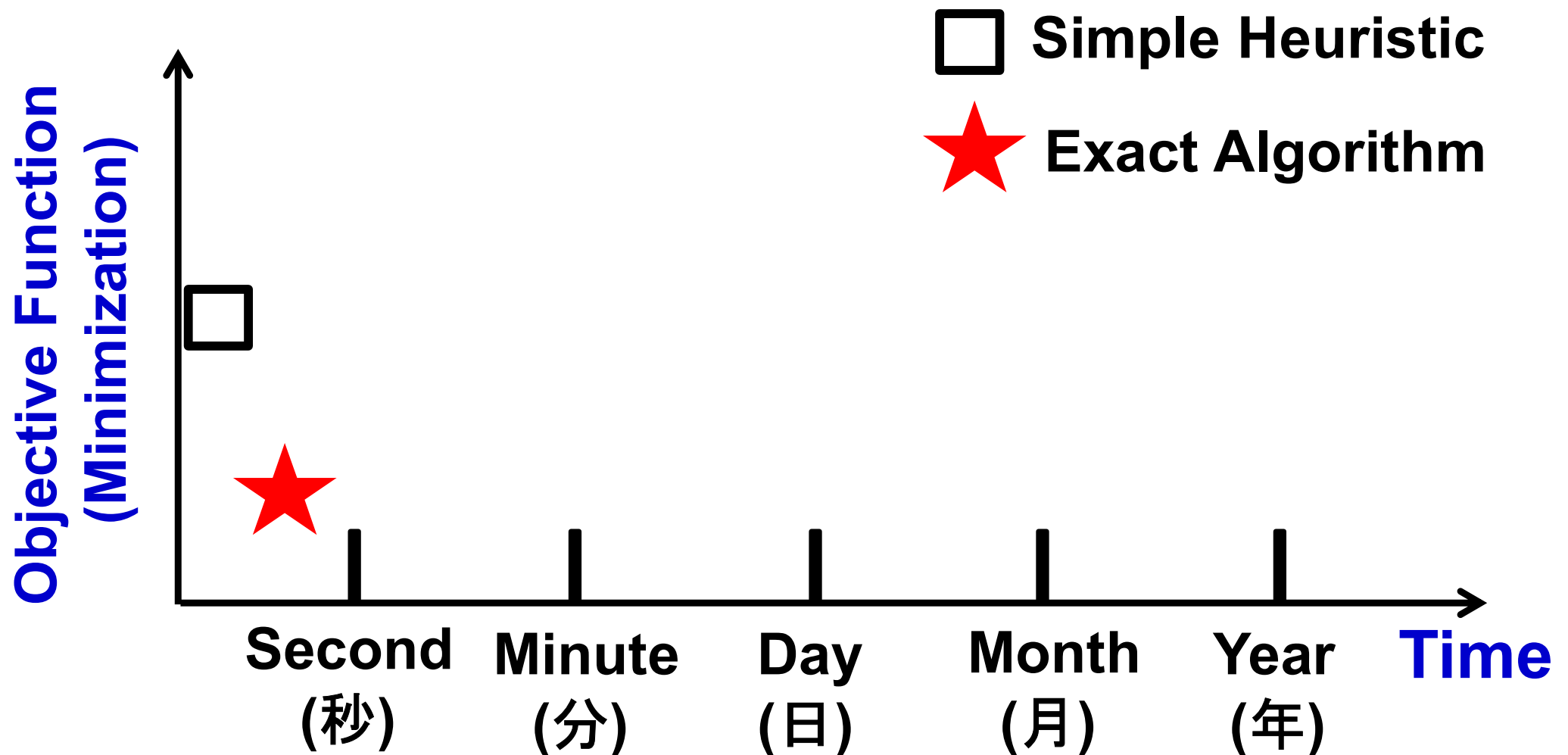
Example 5: Evaluate each research topic and sort them from the most difficult topic to the simplest topic. Evaluate each student and sort them from the best student to the worst student. Then we can assign the 20 topics to the 20 students depending on the topic difficulty.

Advantage: Very fast

Discussions about Computation Load

For combinatorial optimization problems:

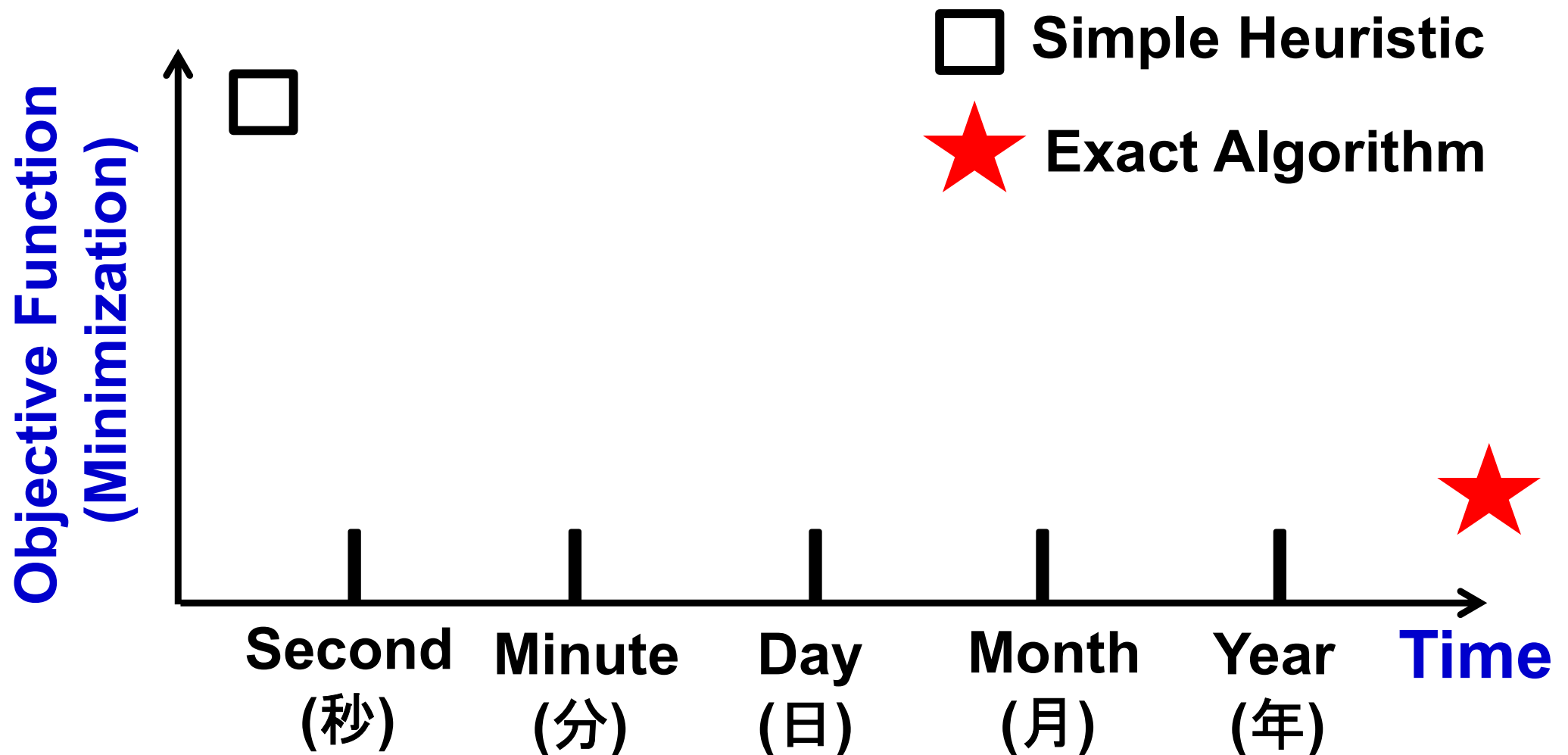
For Small Problem (Assignment of 4 topics)



Discussions about Computation Load

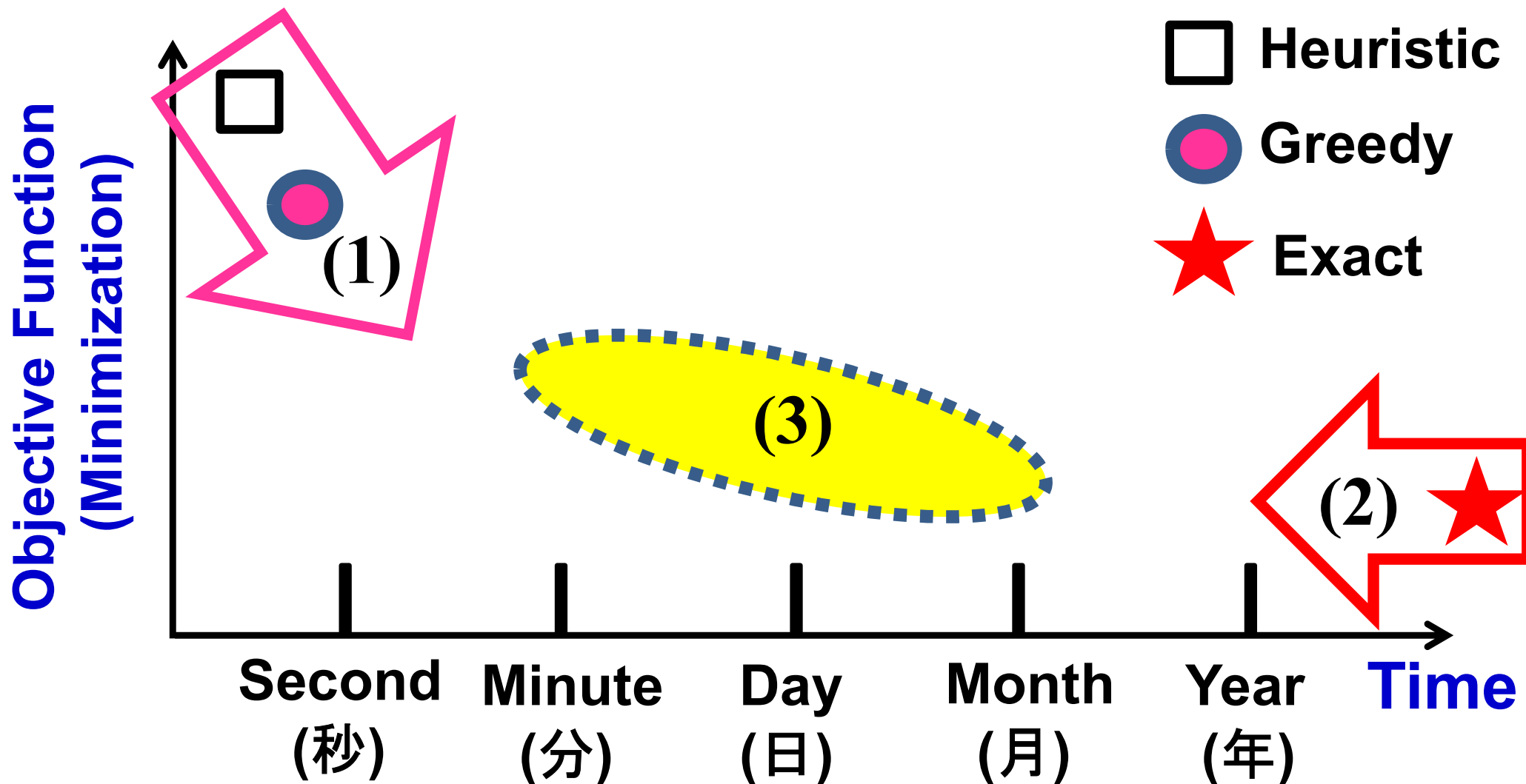
For combinatorial optimization problems:

For Large Problem (Assignment of 20 topics)



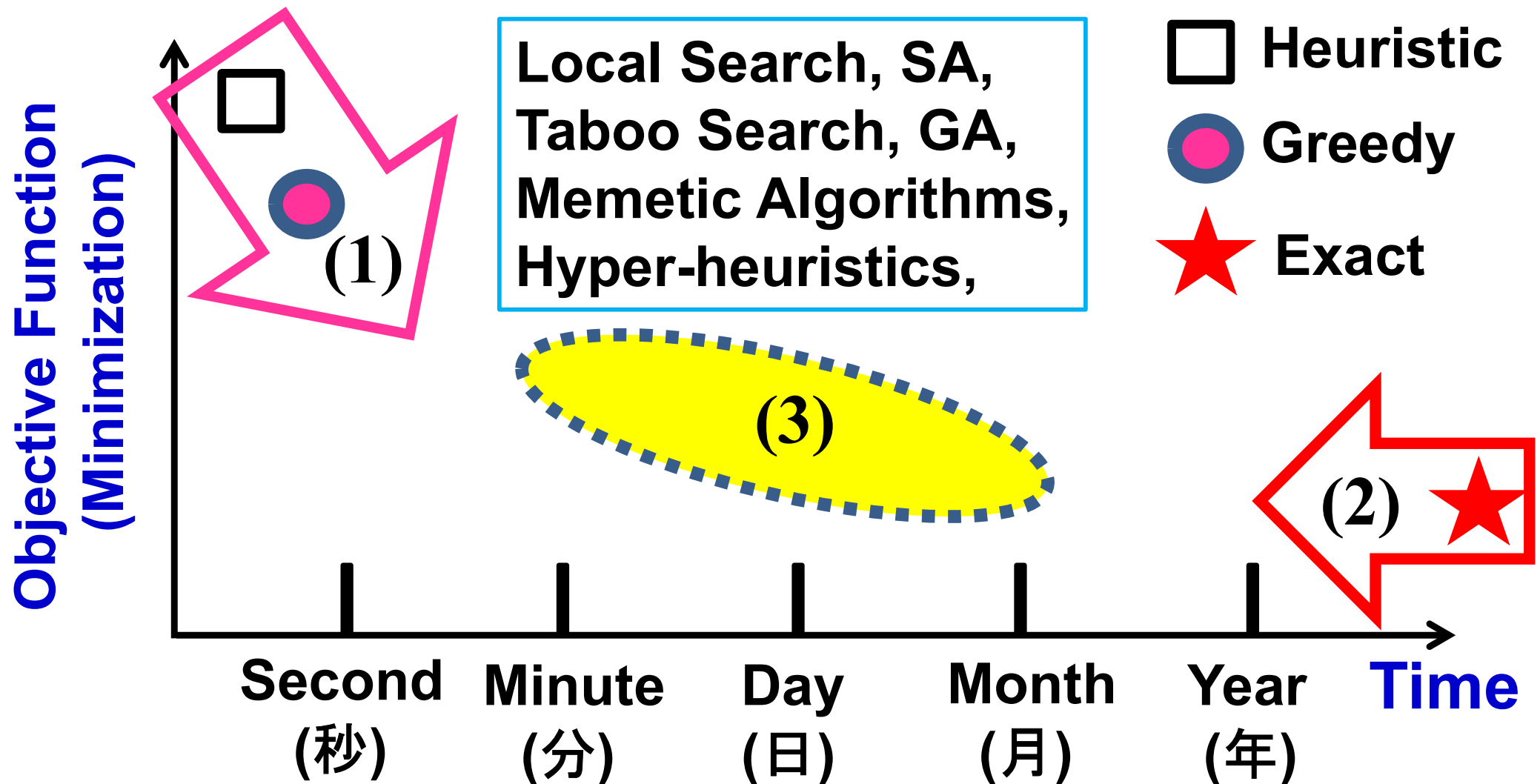
Research Directions in Optimization

- (1) Accuracy improvement
- (2) Efficiency improvement
- (3) Huge gap between (1) and (2)



Focus of this course: Huge gap between (1) and (2)

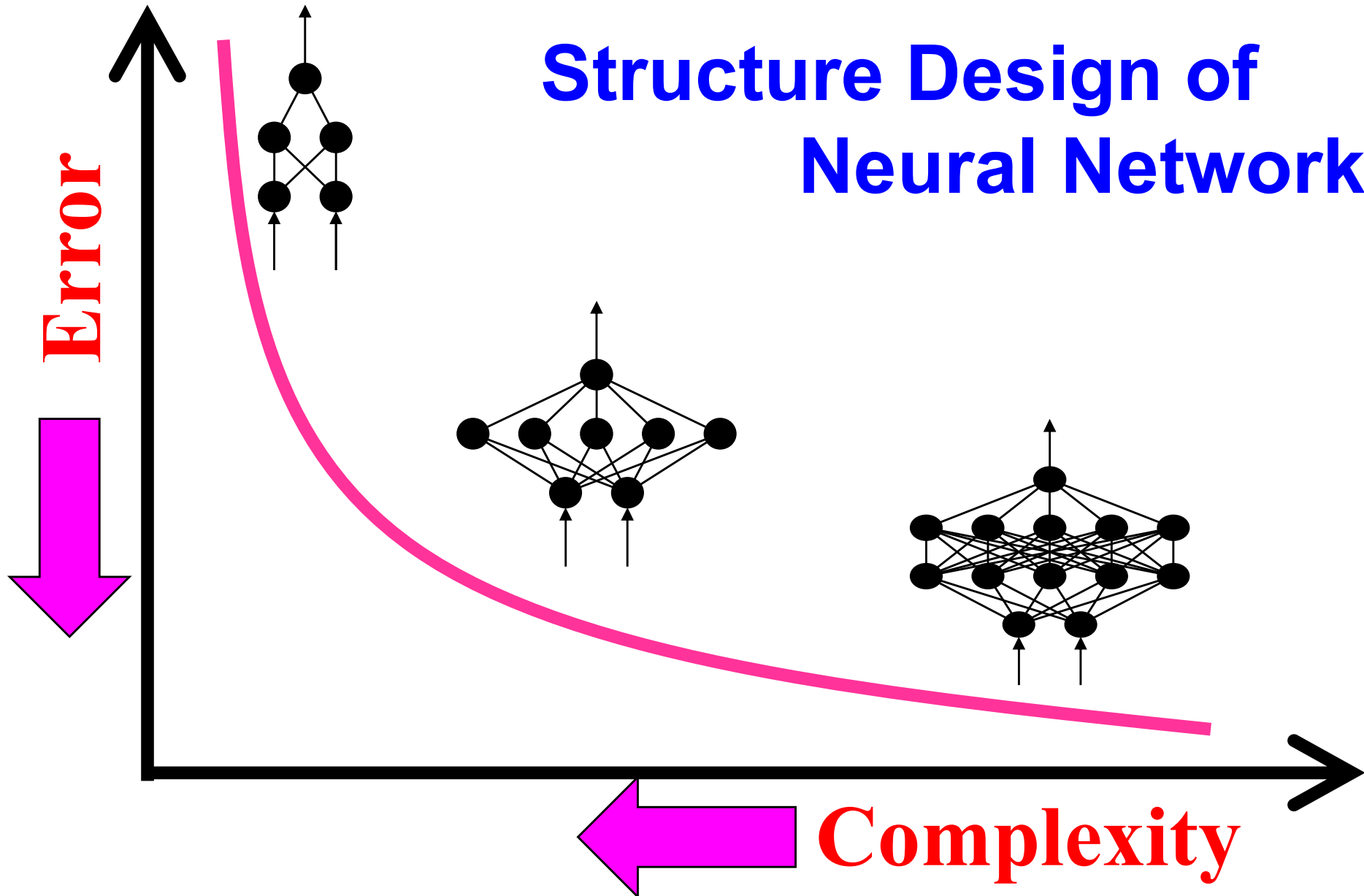
- We have much more time than (1): One solution.
- We do not have enough time for (2): All solutions.



The Other Focus: Multi-Objective Optimization

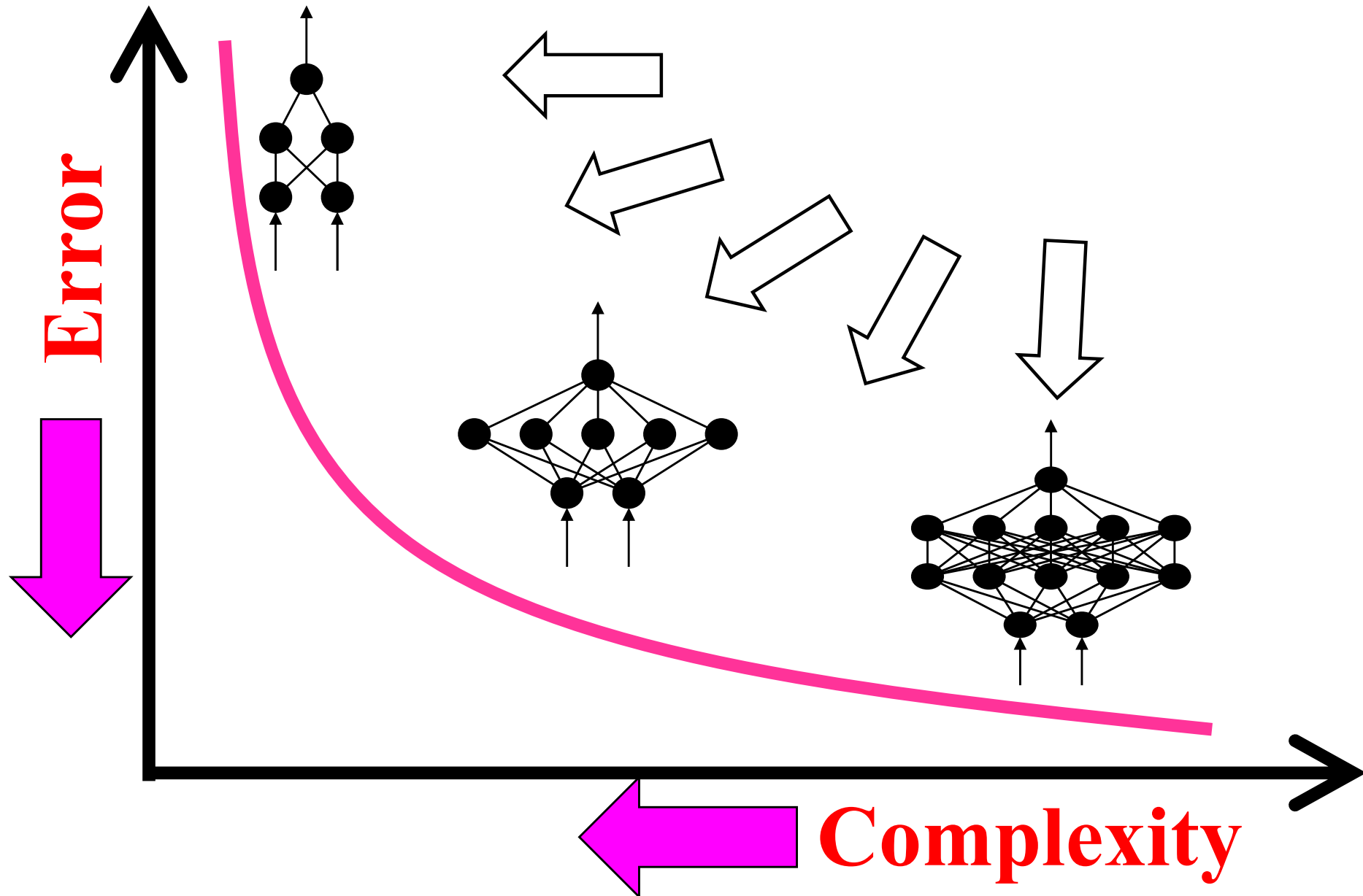
Almost all problems have multiple objectives

Structure Design of Neural Networks



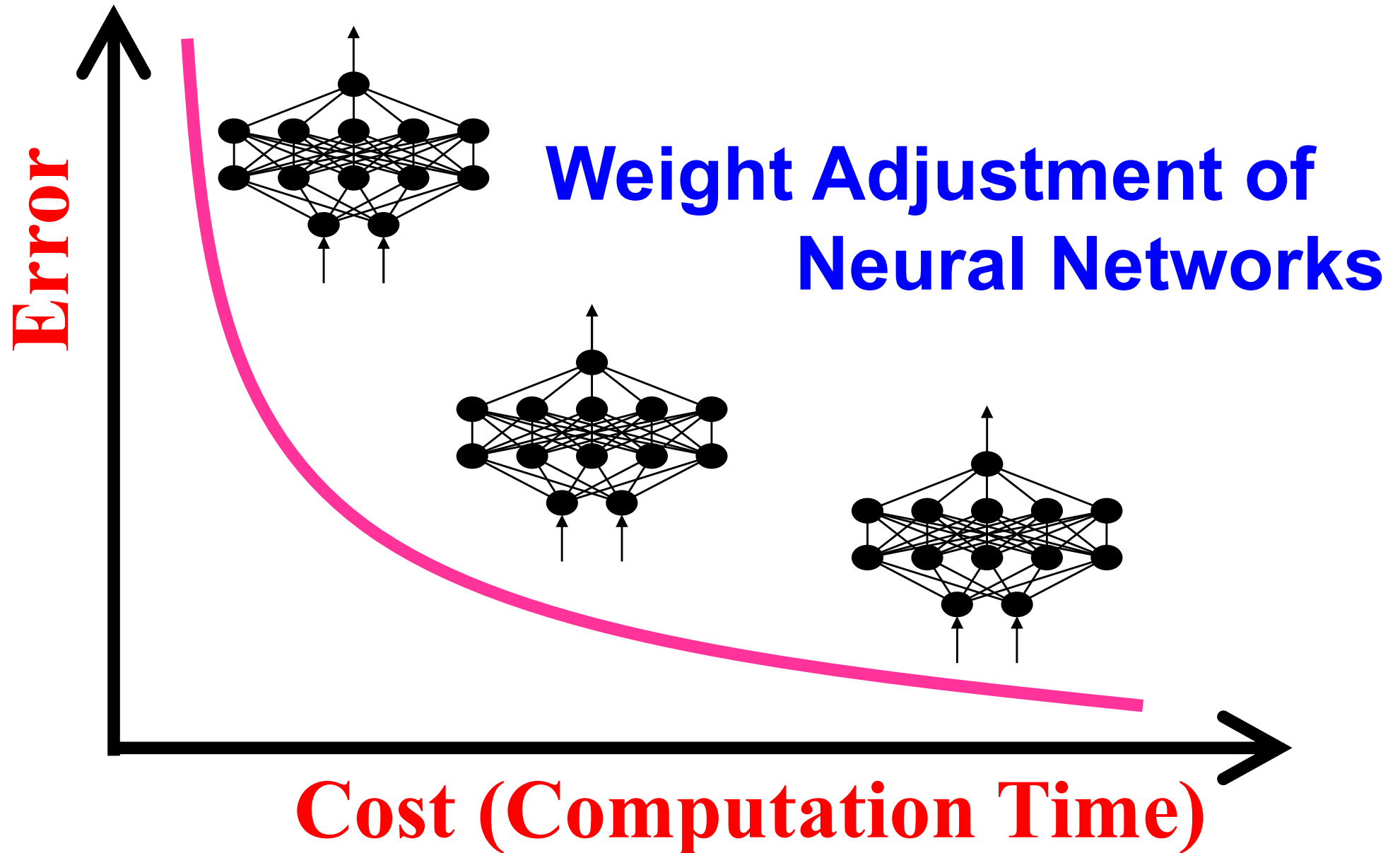
The Other Focus: Multi-Objective Optimization

Almost all problems have multiple objectives



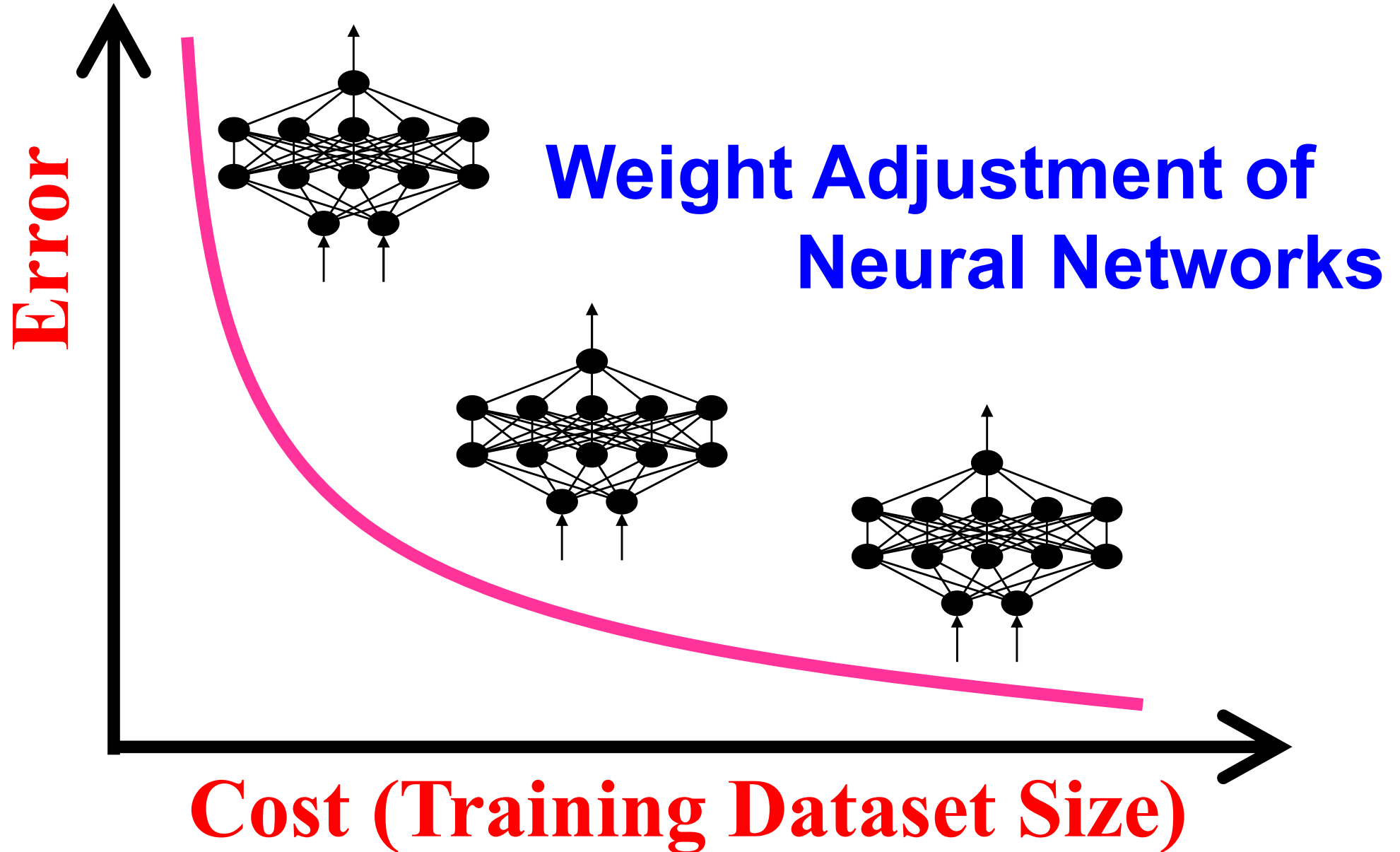
The Other Focus: Multi-Objective Optimization

Almost all problems have multiple objectives



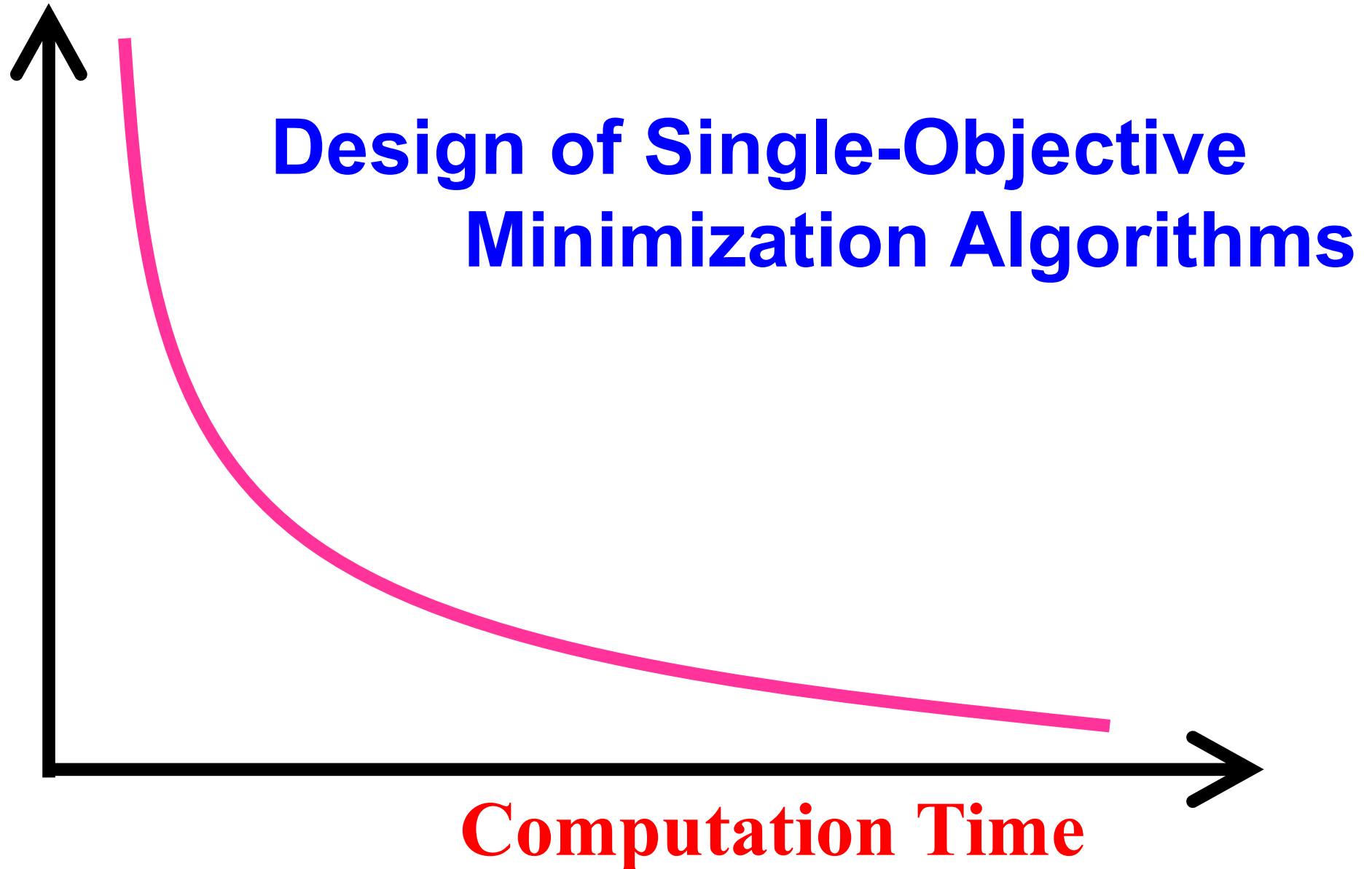
The Other Focus: Multi-Objective Optimization

Almost all problems have multiple objectives



The Other Focus: Multi-Objective Optimization

Almost all problems have multiple objectives



The Other Focus: Multi-Objective Optimization

Almost all problems have multiple objectives

4B · FRIDAY, JUNE 12, 2009 · USA TODAY

Test Drive

Every Friday

Audi diesel has power, elegance

Too bad it doesn't have
more space inside

The Audi Q7 TDI is a recently launched diesel-power variant of the brand's popular (by Audi's low-volume standards) big SUV.

TDI stands for turbocharging and direct injection of the fuel. Turbocharging lets an engine accelerate more quickly. Direct injection of the fuel improves power and cuts pollution. Programmed just so, as it apparently is in the Q7, direct injection can minimize a diesel's signature rocks-in-a-tin-can sound.

Diesels can get 25% to



What stands out

► **Powerful:** At low speed where

2009 Audi Q7 T

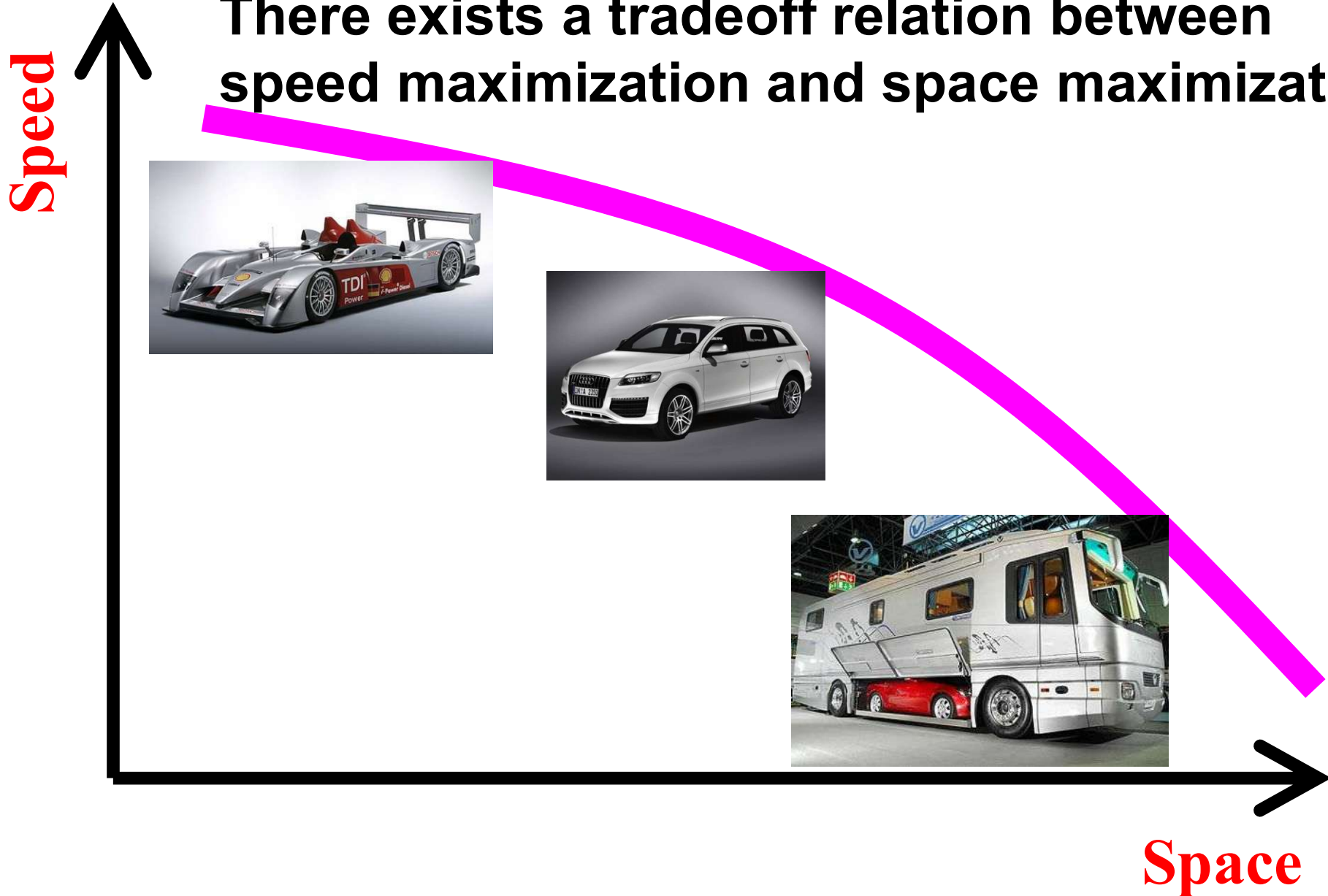
► **What?** Diesel-engine version of the brand's large four-door seven-passenger crossover SUV

truck-t
erwise

The Other Focus: Multi-Objective Optimization

Almost all problems have multiple objectives

There exists a tradeoff relation between speed maximization and space maximization.



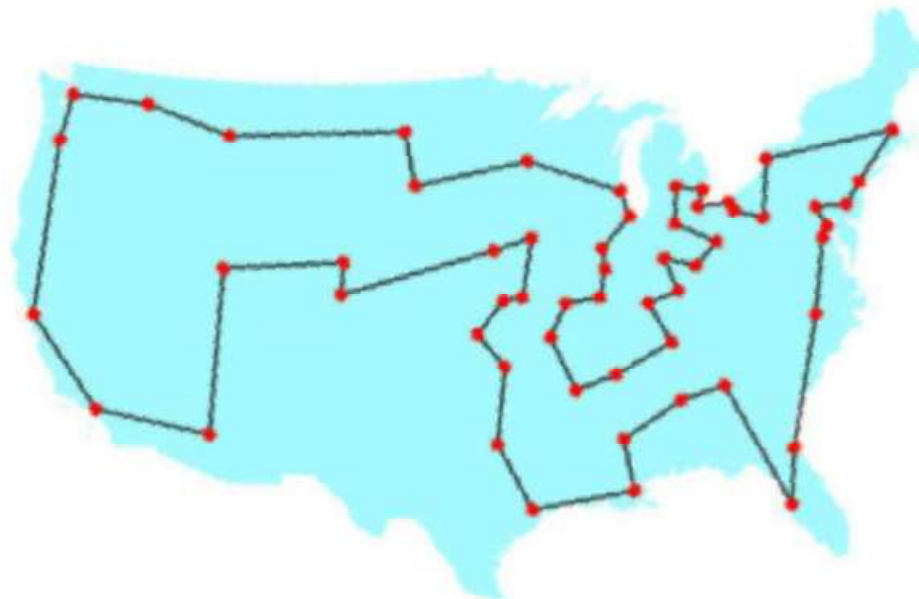
Travelling Salesman Problem (TSP): To find the shortest tour to visit all cities and return to the start city.

Input: City set: n cities ($i = 1, 2, \dots, n$)

Distance between each pair of cities (i, j): $d_{ij} = d_{ji}$

Objective: Minimization of a tour length starting from a city, visiting all cities and returning to the start city.

Output: Tour with the minimum distance



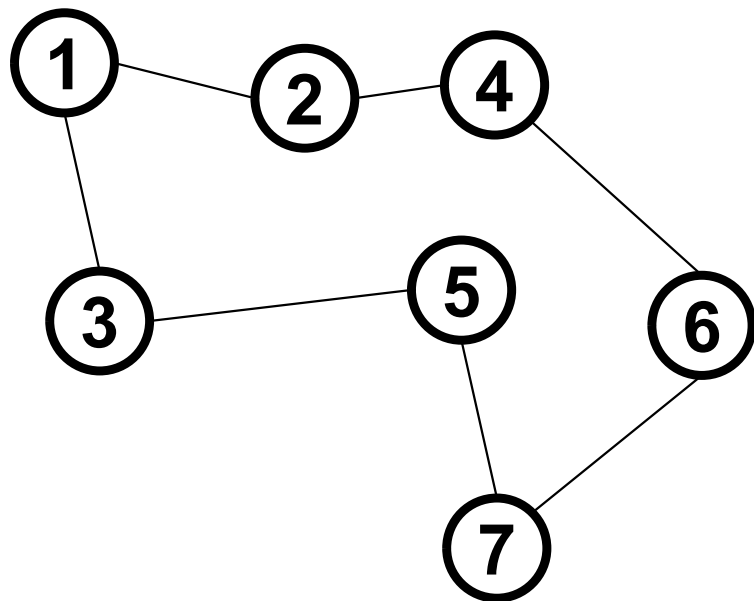
Travelling Salesman Problem (TSP): To find the shortest tour to visit all cities and return to the start city.

Problem Size: The number of cities n (e.g., 100 cities)

Search Space Size: The number of possible combinations

The total number of permutations of n cities:

$$n! = n(n-1)(n-2) \dots 1$$

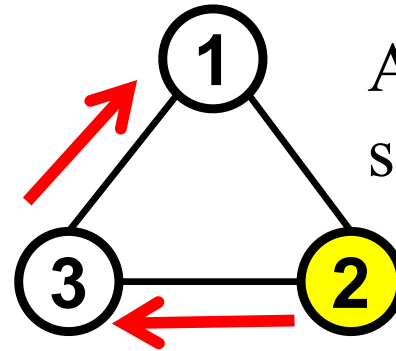
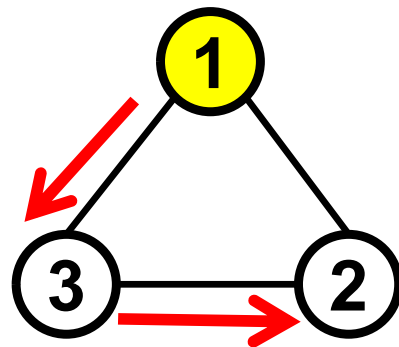
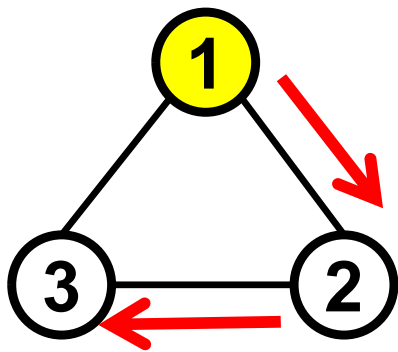


Travelling Salesman Problem (TSP): To find the shortest tour to visit all cities and return to the start city.

Problem Size: The number of cities n (e.g., 100 cities)

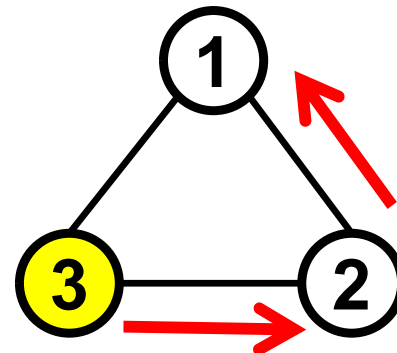
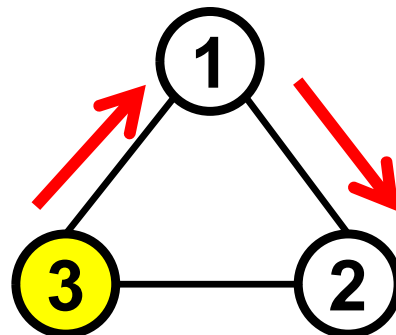
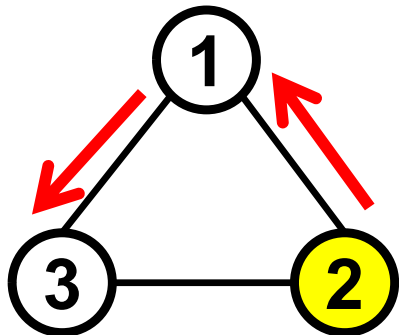
Search Space Size: The number of possible combinations

When $n = 3$ (3 cities): The number of permutations of 3 cities is $n! = 6$



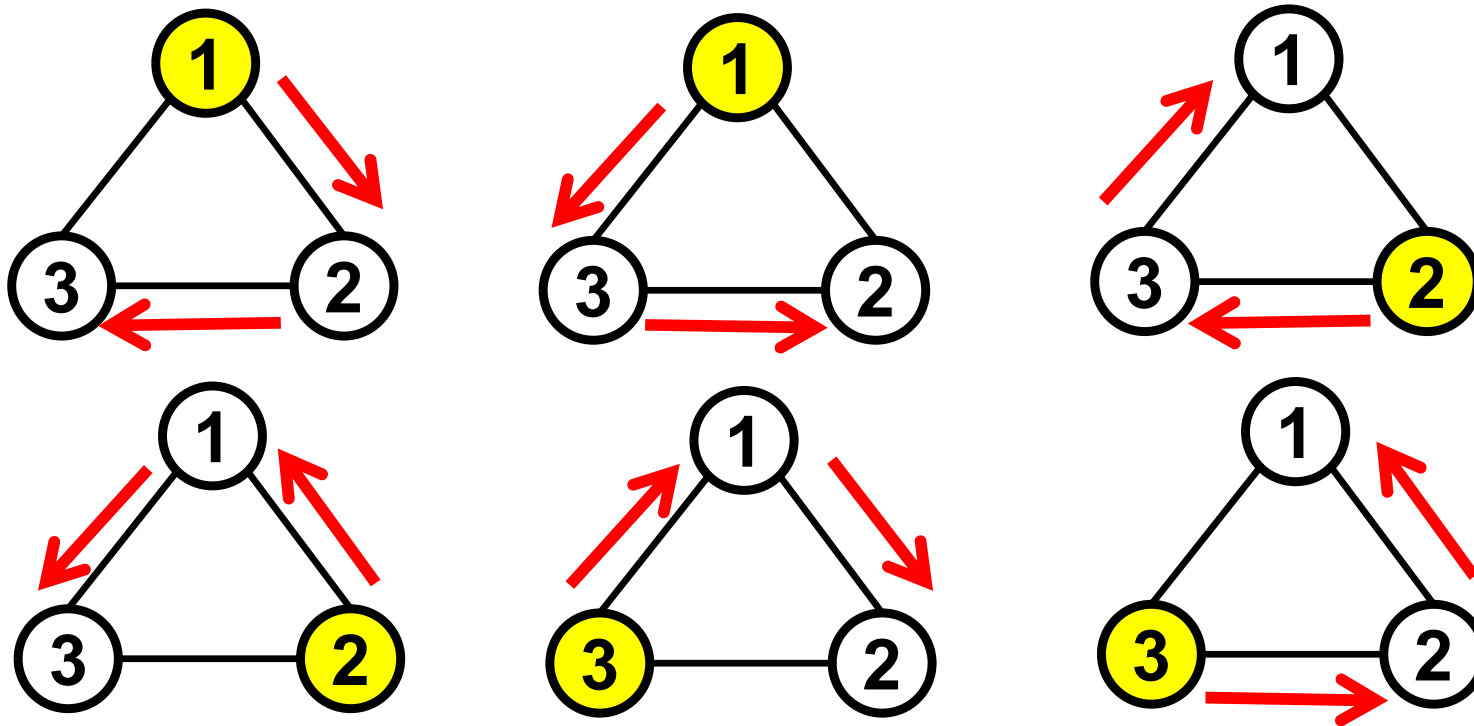
All of them are the same solution:

A single tour.



When $n = 3$ (3 cities): $n! = 6 \Rightarrow$ **One tour.**

- We can specify one city as the first city:
permutations of $(n-1)$ cities. $n! \Rightarrow (n-1)!$
- One permutation has the same tour length as its reverse order:
 $Length(1231) = Length(1321): (n-1)! \Rightarrow (n-1)!/2$
- The search space size: $(n-1)!/2$



How large is $(n-1)!/2$?

Example 5: Assignment of 20 research topics to 20 students. The total number of different combinations is $20 \times 19 \times \dots \times 1 = 2.4 \times 10^{18}$. Even if we can use your **one-million/second** computer, we need **77000 years** to examine all combinations:

20! combinations: one-million/second ==> 77000 years

22-City TSP with $21!/2$ combinations ==> 0.8 million years

10-City TSP with $9!/2$ combinations ==> 0.18 seconds

11-City TSP with $10!/2$ combinations ==> 1.8 seconds

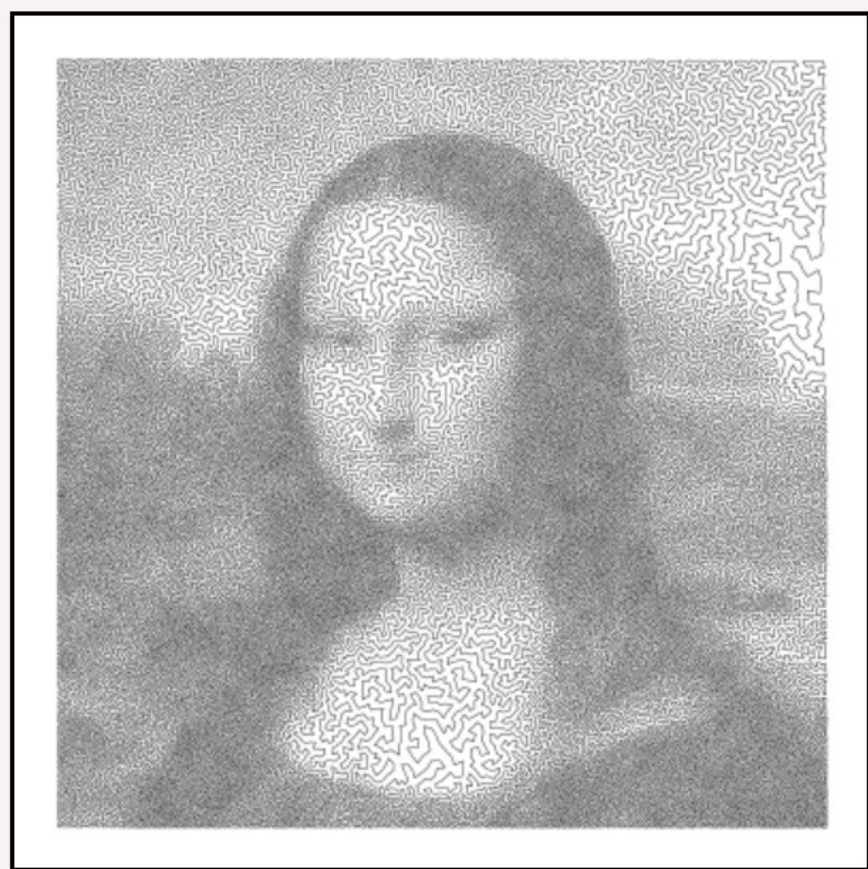
12-City TSP with $11!/2$ combinations ==> 20 seconds

13-City TSP with $12!/2$ combinations ==> 4.0 minutes

14-City TSP with $13!/2$ combinations ==> 52 minutes

Mona Lisa TSP Challenge 100,000 Cities

<http://www.math.uwaterloo.ca/tsp/data/ml/monalisa.html>



Robert Bosch, February 2009
mona-lisa100K.tsp

The current best known results:

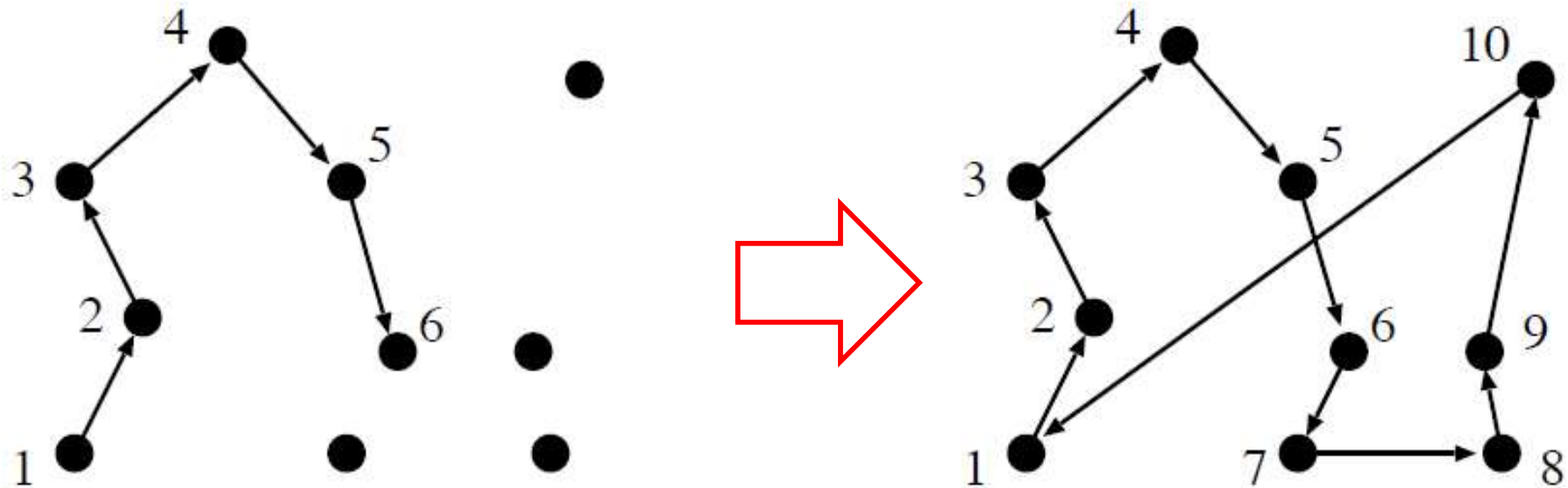
Tour: 5,757,191 Gap: 107 (0.0019%)

The tour was found on March 17, 2009, by Yuichi Nagata. The bound gives a value B such that no tour has length less than B ; this bound was found on July 27, 2012, with the Concorde code. The Gap number is the gap in our knowledge, that is, the difference between the length of the tour and the value of the bound.

It has been over three years since Yuichi Nagata produced the best-known tour. To help perk up interest in searching for an even better solution, we are offering a \$1,000 prize to the first person to find a tour shorter than 5,757,191.

Nearest Neighbor Greedy Method

1. Arbitrarily select a starting city.
2. Move from the current city to the nearest city among the remaining cities.



Nearest Neighbor Greedy Method

Computation Load

(1) Determination of the 2nd city

- Distance calculation from the current city to $(n-1)$ cities.
- Search for the minimum distance among $(n-1)$ distances.
This needs $(n-2)$ comparisons.

(2) Determination of the 3rd city

- Distance calculation from the current city to $(n-2)$ cities.
- Search for the minimum distance among $(n-2)$ distances.
This needs $(n-3)$ comparisons.

Total Calculation: Order of n^2

- $n(n-1)/2$ distance calculations
- $(n-1)(n-2)/2$ comparisons

How Large?

$n = 22$ (Examination of all solutions: 0.8 million years)

- $n(n-1)/2$ distance calculations: 231 calculations
- $(n-1)(n-2)/2$ comparisons: 210 comparisons

$n = 100$

- $n(n-1)/2$ distance calculations: 4950 calculations
- $(n-1)(n-2)/2$ comparisons: 4851 comparisons

$n = 100,000$ (Mona Lisa)

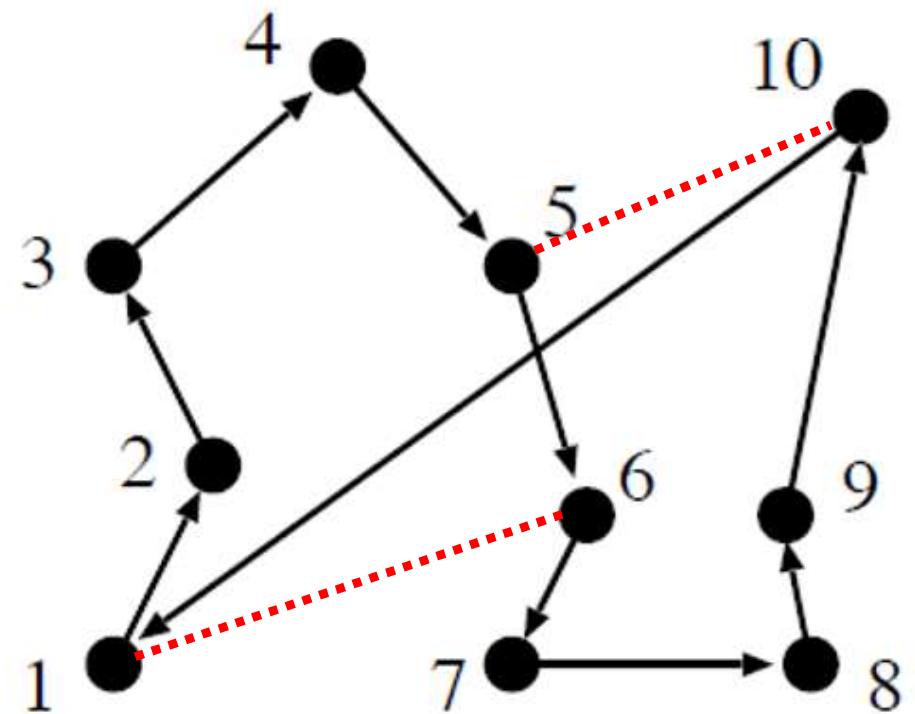
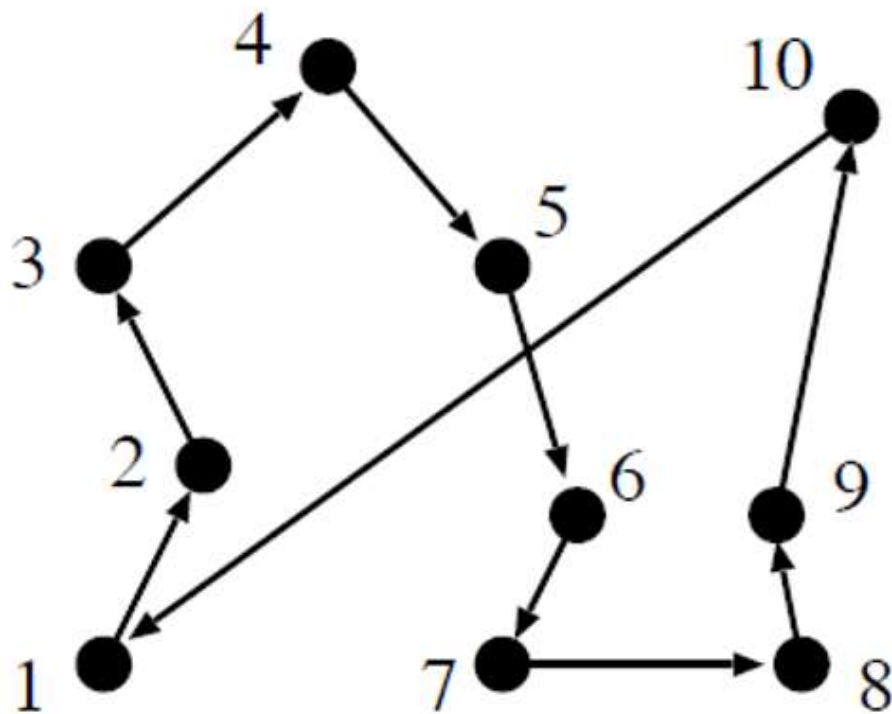
- $n(n-1)/2$ distance calculations: 4,999,950,000 calculations
- $(n-1)(n-2)/2$ comparisons: 4,999,850,001 comparisons

4,999,950,000 calculations $\implies$ 1.4 hours.

4,999,850,001 comparisons $\implies$ 1.4 hours

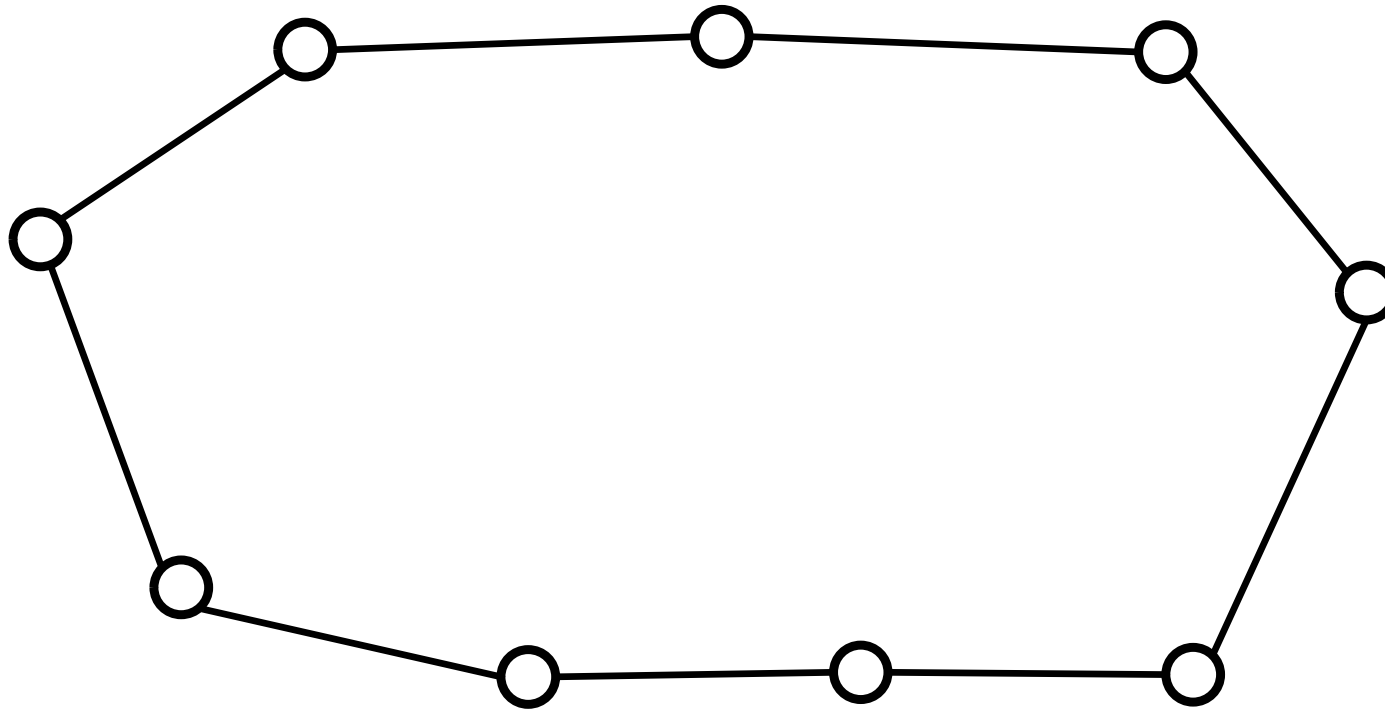
The obtained tour is not optimal

If a tour has a cross-point (i.e., if two edges intersect with each other), it is not optimal. Geometrically it is clear in the following example that the modified tour with the dashed red lines is shorter than the original tour.



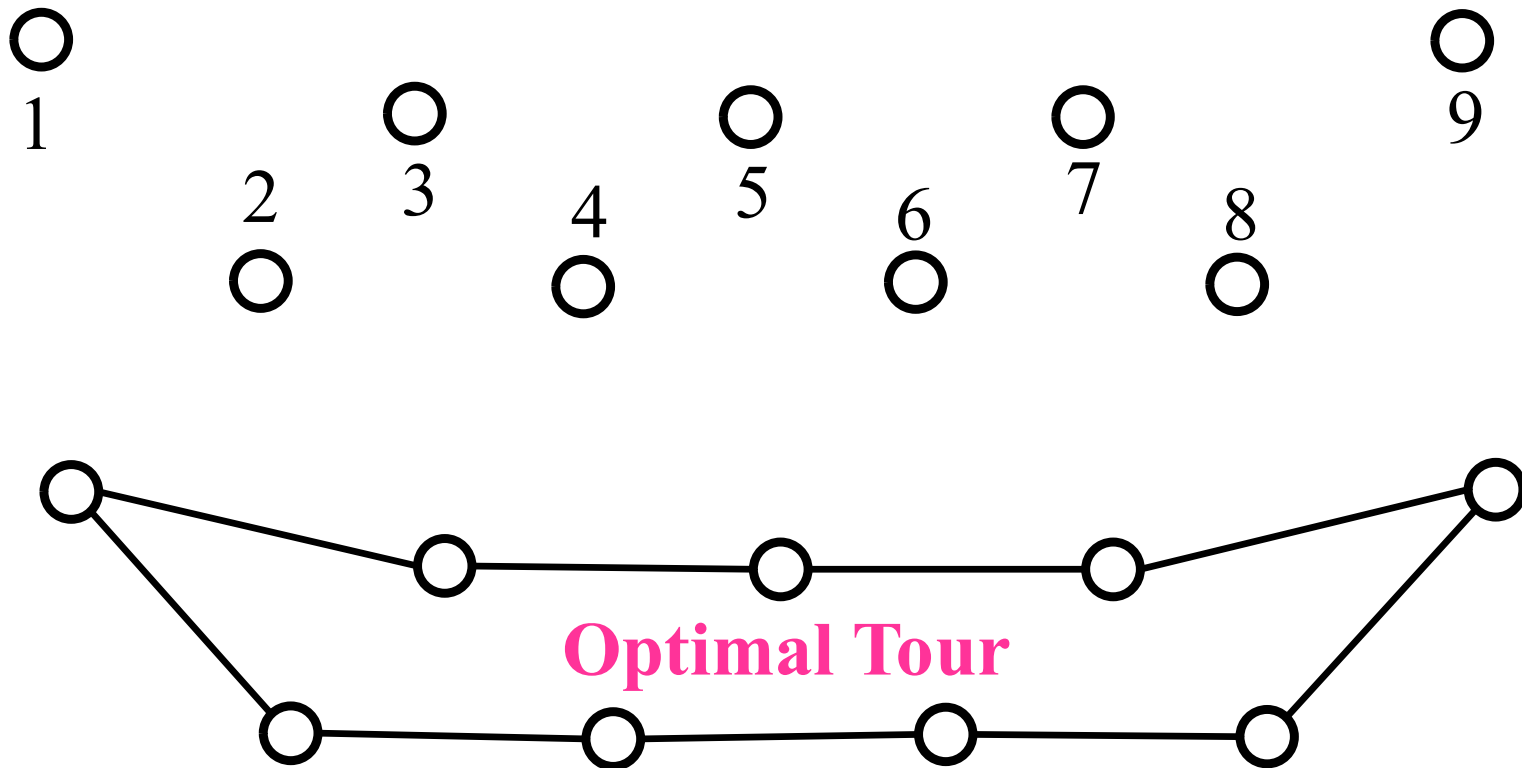
Use of Simple Examples

Examples are helpful to understand the greedy algorithm (and any algorithm). Create an example (problem instance) where the optimal solution is always obtained by the nearest neighbor greedy algorithm independent of the choice of a starting city.



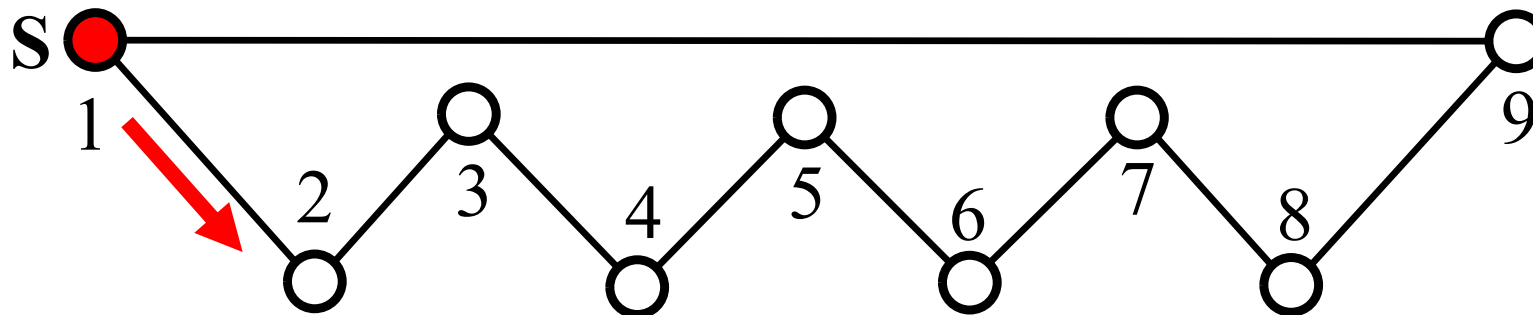
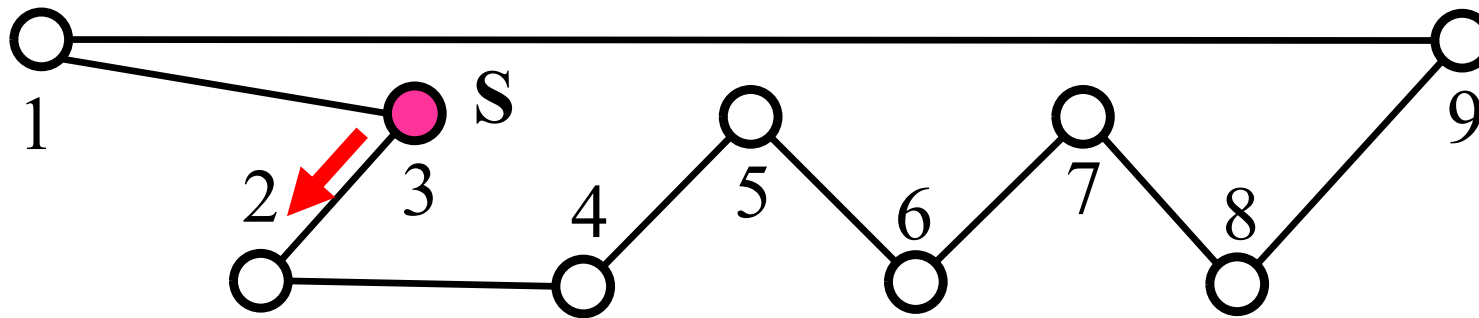
Use of Simple Examples

Create an example (problem instance) where the obtained tour by the nearest neighbor greedy algorithm is longer than the optimal tour independent of the choice of a starting city.



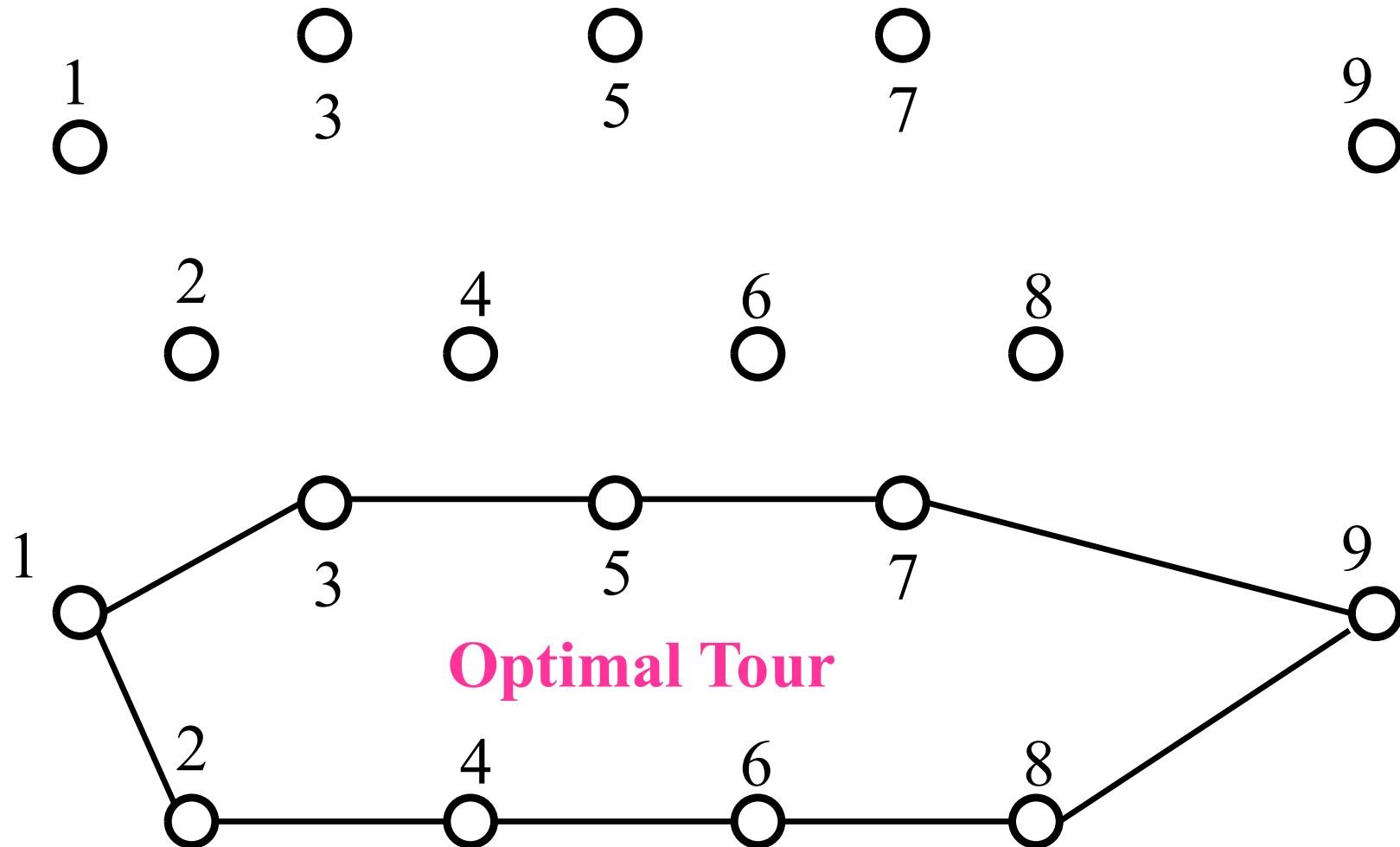
Use of Simple Examples

Create an example (problem instance) where the obtained tour by the nearest neighbor greedy algorithm is longer than the optimal tour independent of the choice of a starting city.



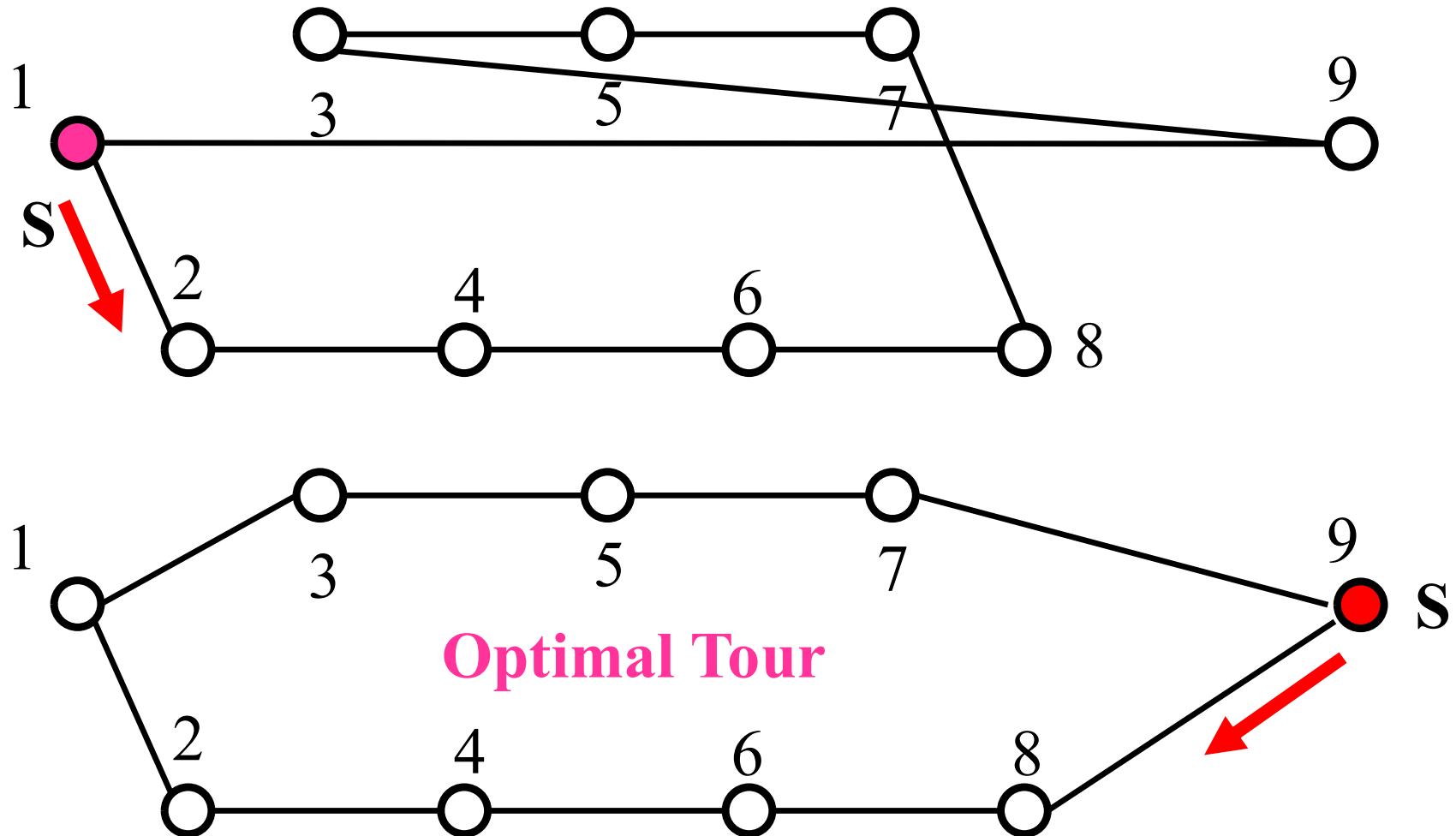
Use of Simple Examples

Create an example (problem instance) where the optimal tour is obtained by the nearest neighbor greedy algorithm from some starting cities but not obtained from the other starting cities.



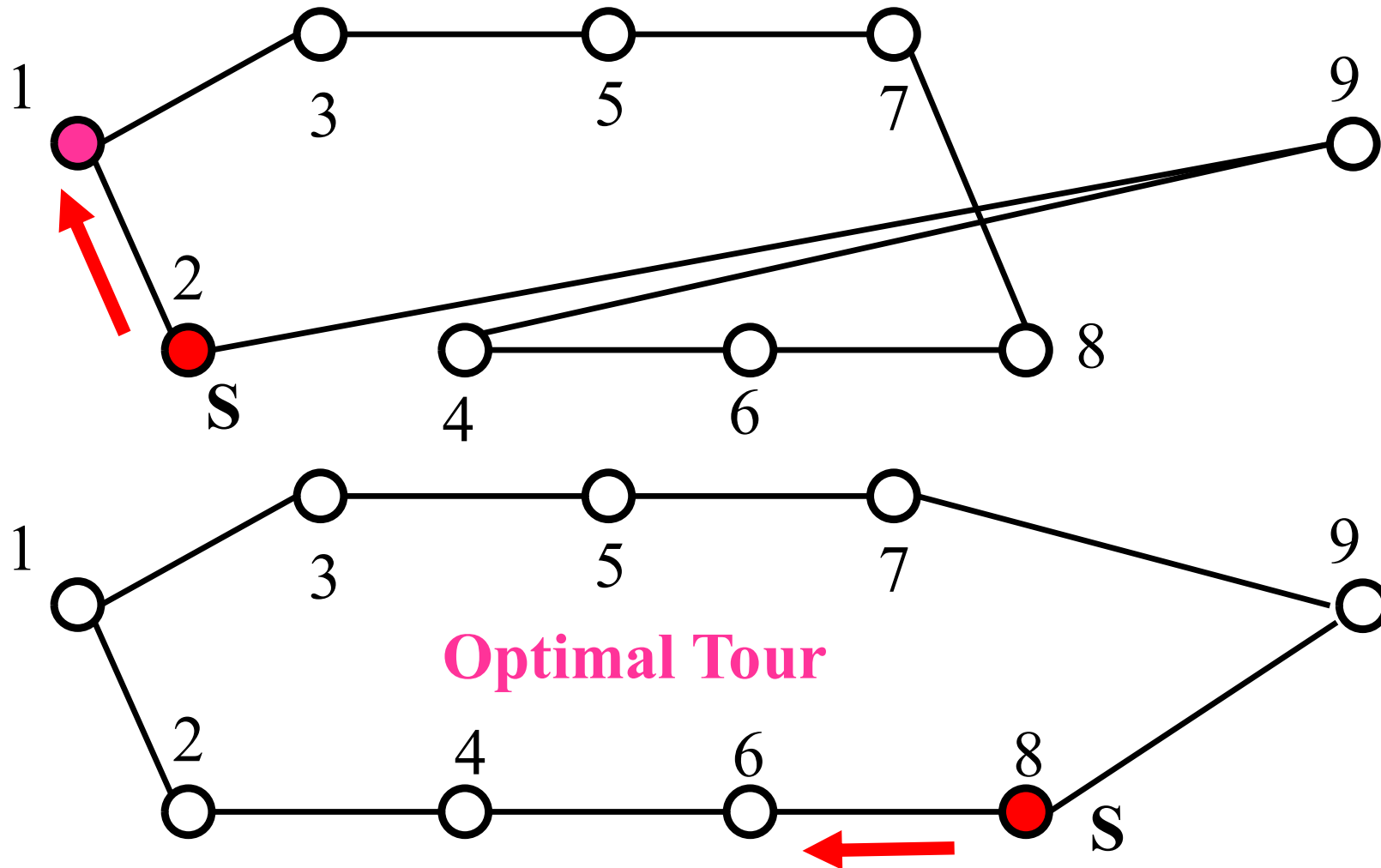
Use of Simple Examples

Create an example (problem instance) where the optimal tour is obtained by the nearest neighbor greedy algorithm from some starting cities but not obtained from the other starting cities.



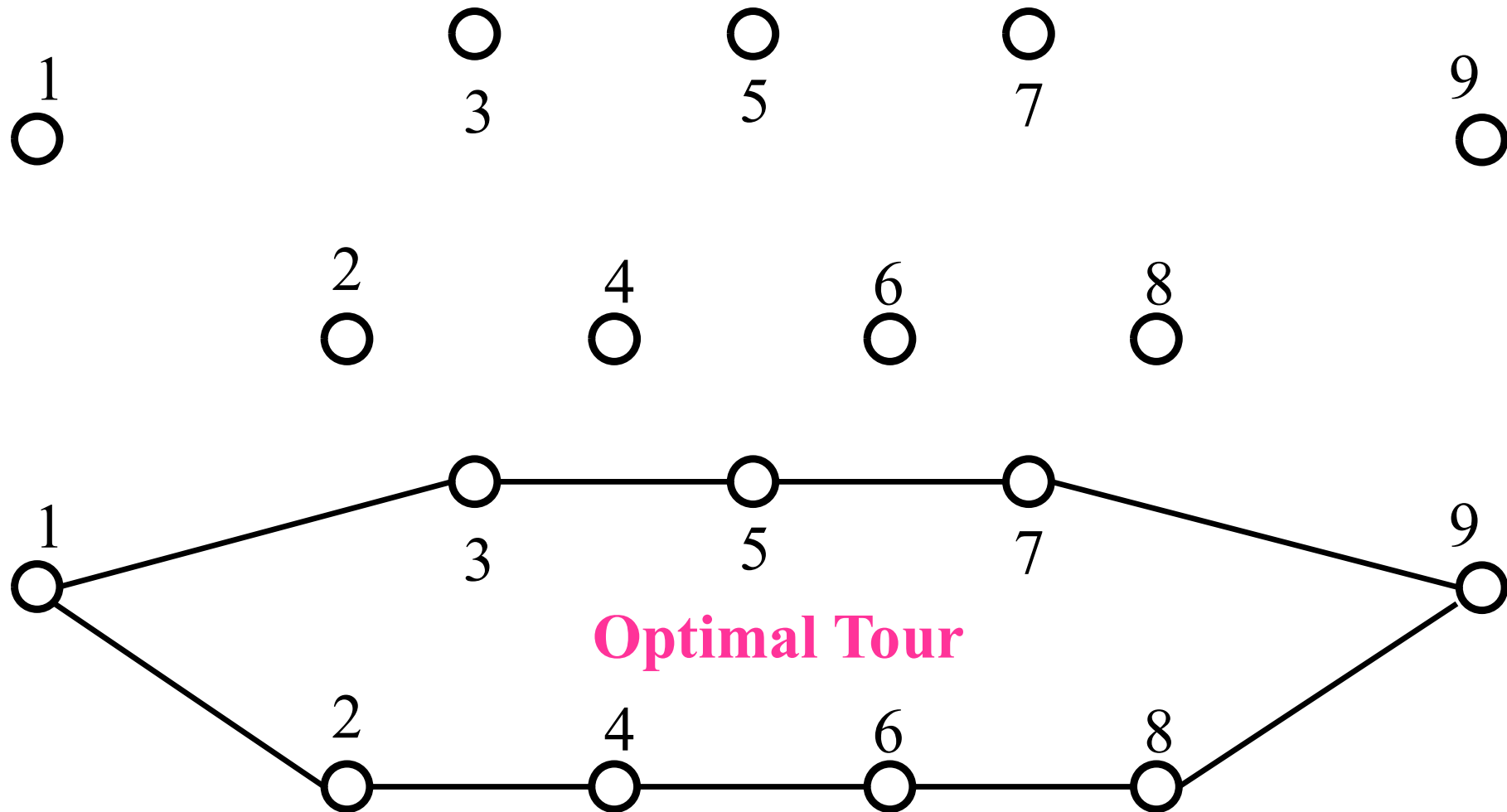
Use of Simple Examples

Create an example (problem instance) where the optimal tour is obtained by the nearest neighbor greedy algorithm from some starting cities but not obtained from the other starting cities.



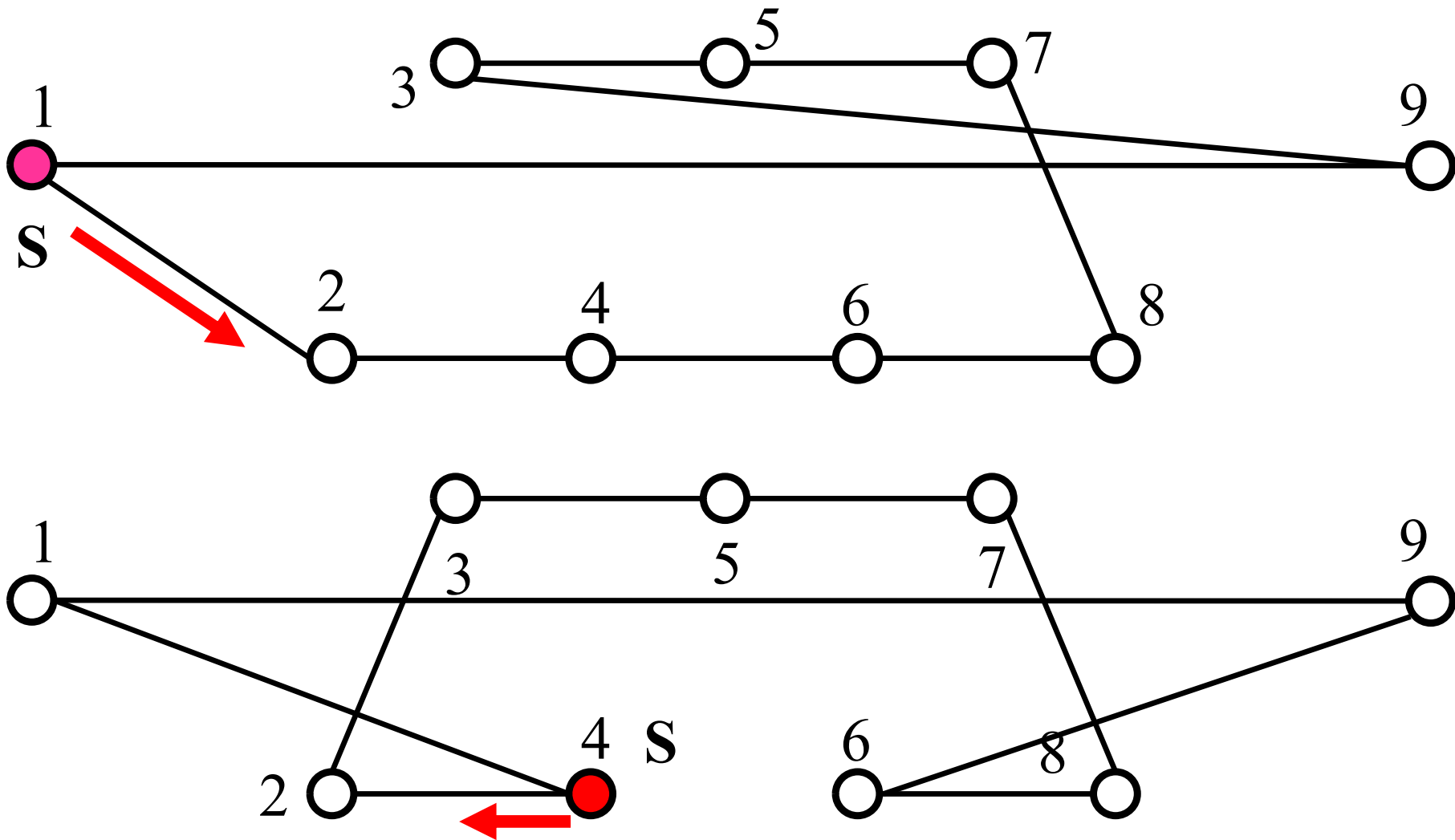
Use of Simple Examples

Create an example (problem instance) where the obtained tour by the nearest neighbor greedy algorithm is longer than the optimal tour independent of the choice of a starting city.



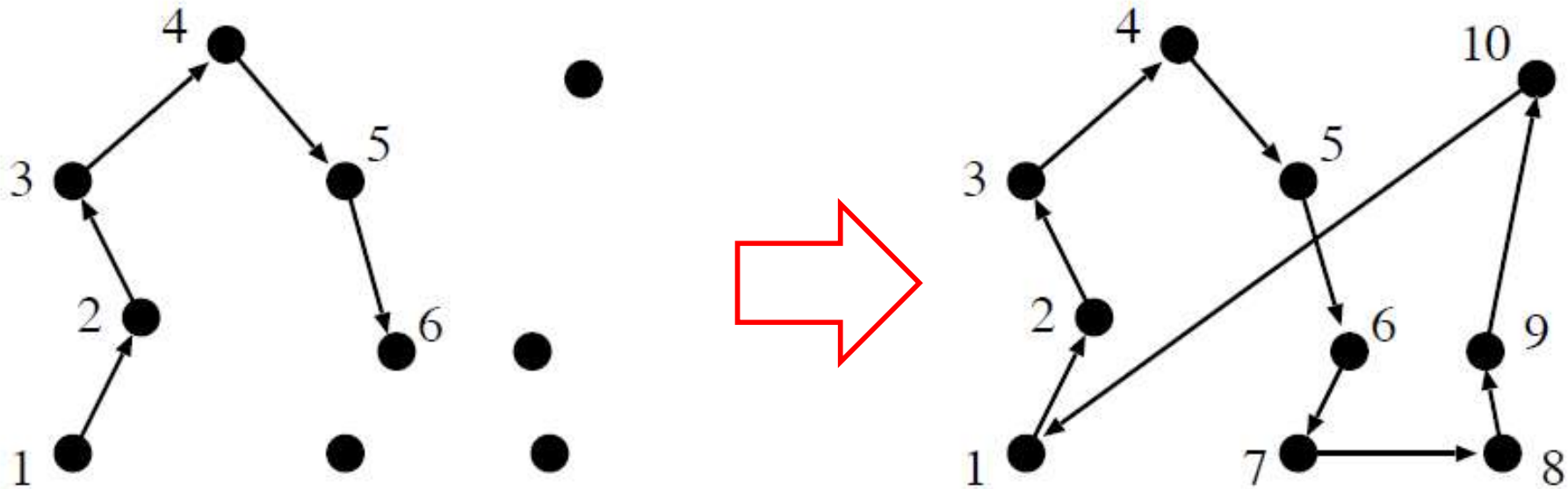
Use of Simple Examples

Create an example (problem instance) where the obtained tour by the nearest neighbor greedy algorithm is longer than the optimal tour independent of the choice of a starting city.



Quick Exercise

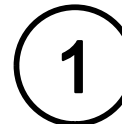
Which is the best starting city for the greedy algorithm?



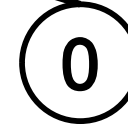
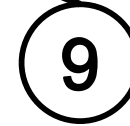
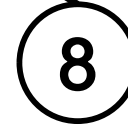
Exact Optimization Branch and Bound

Upper Bound:
Greedy Tour

Start City



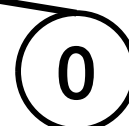
**1st
City**



**2nd
City**

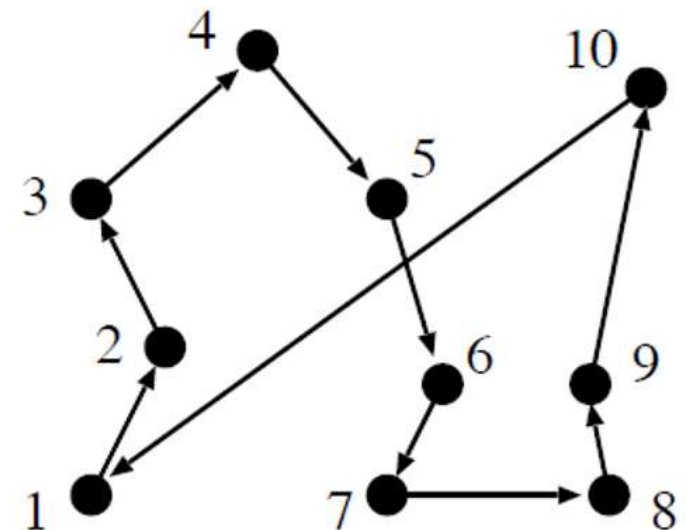


**3rd
City**



Lower Bound:

Current Tour length
Length (1-2-3-4-1)



Implementation Issues.

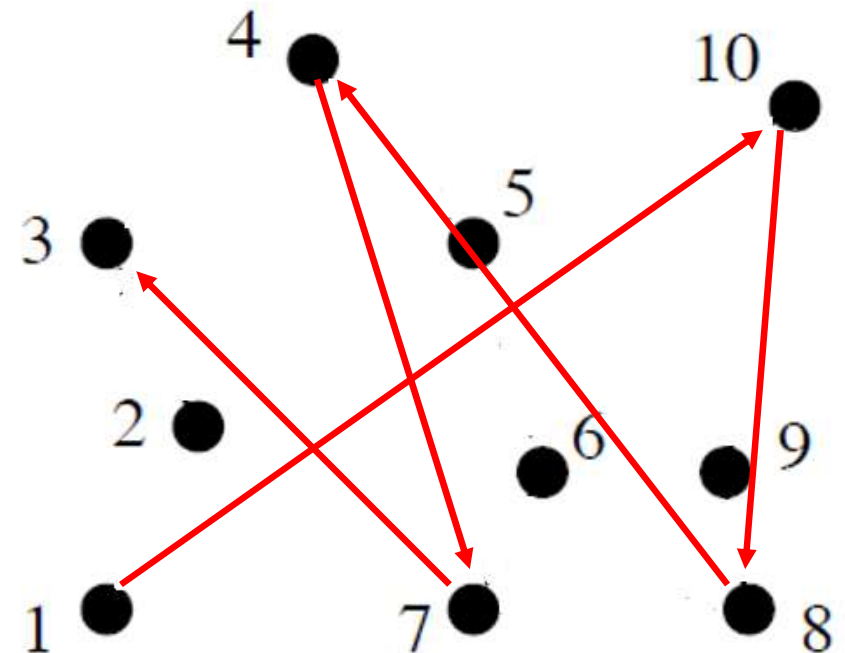
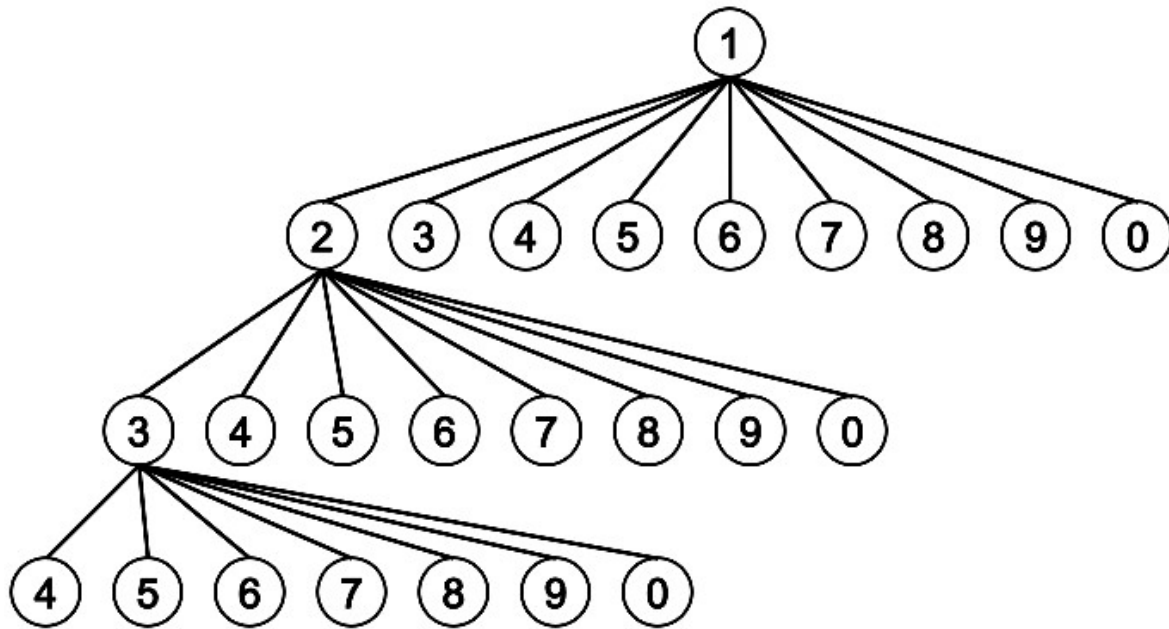
(1) How to calculate the lower bound of each node.

Some tours may be easily identified as bad tours by calculating a part of them (e.g., tours starting with “1 10 8 4 7 3”).

(2) How to choose the node for the next branching.

(3) How to check the same tours for efficient search.

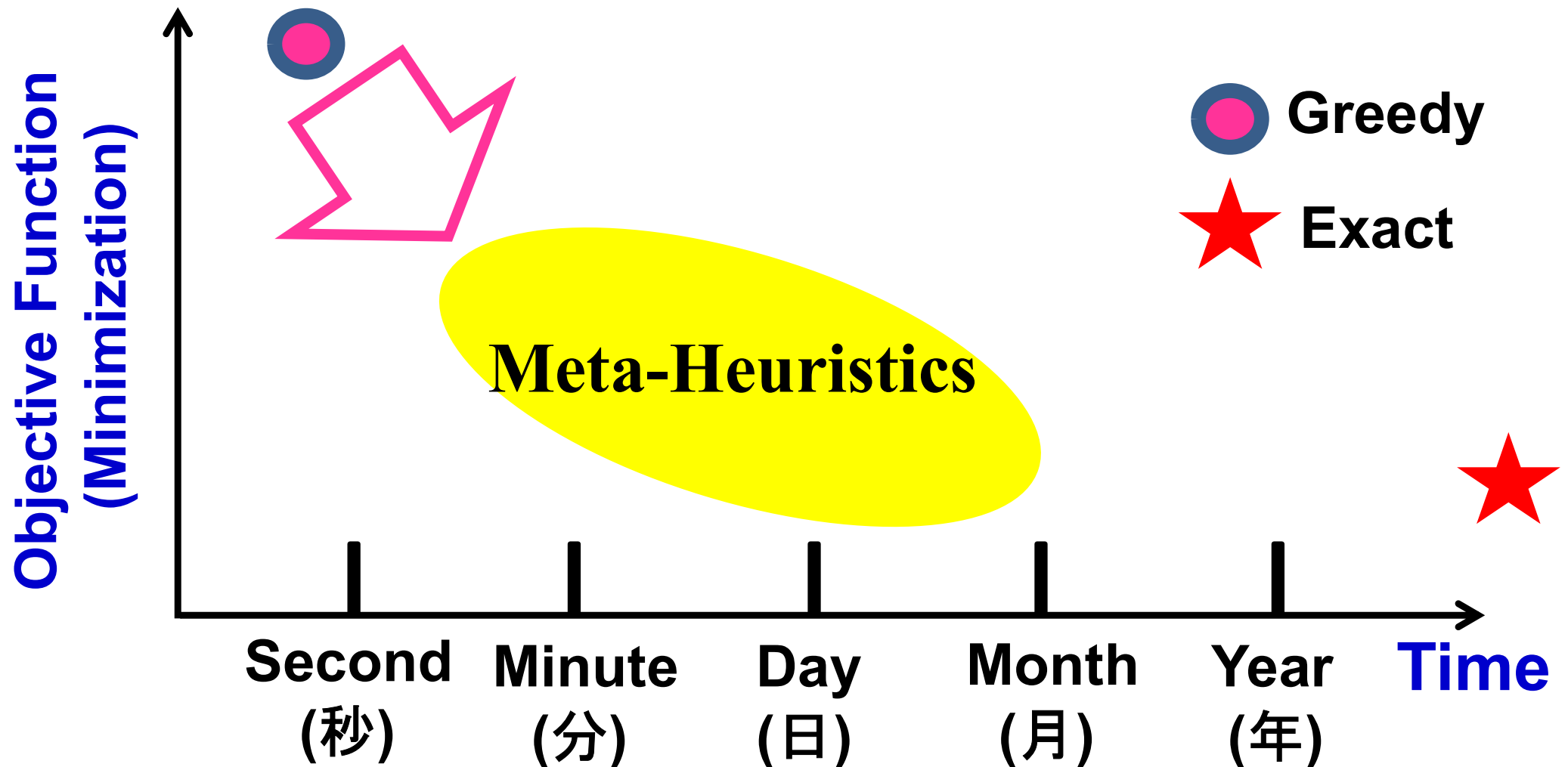
For example, “1 2 ... 9 10 1” is the same as “1 10 9 ... 2 1”.



Two Optimization Methods

Greedy: Very fast, Results can be very bad.

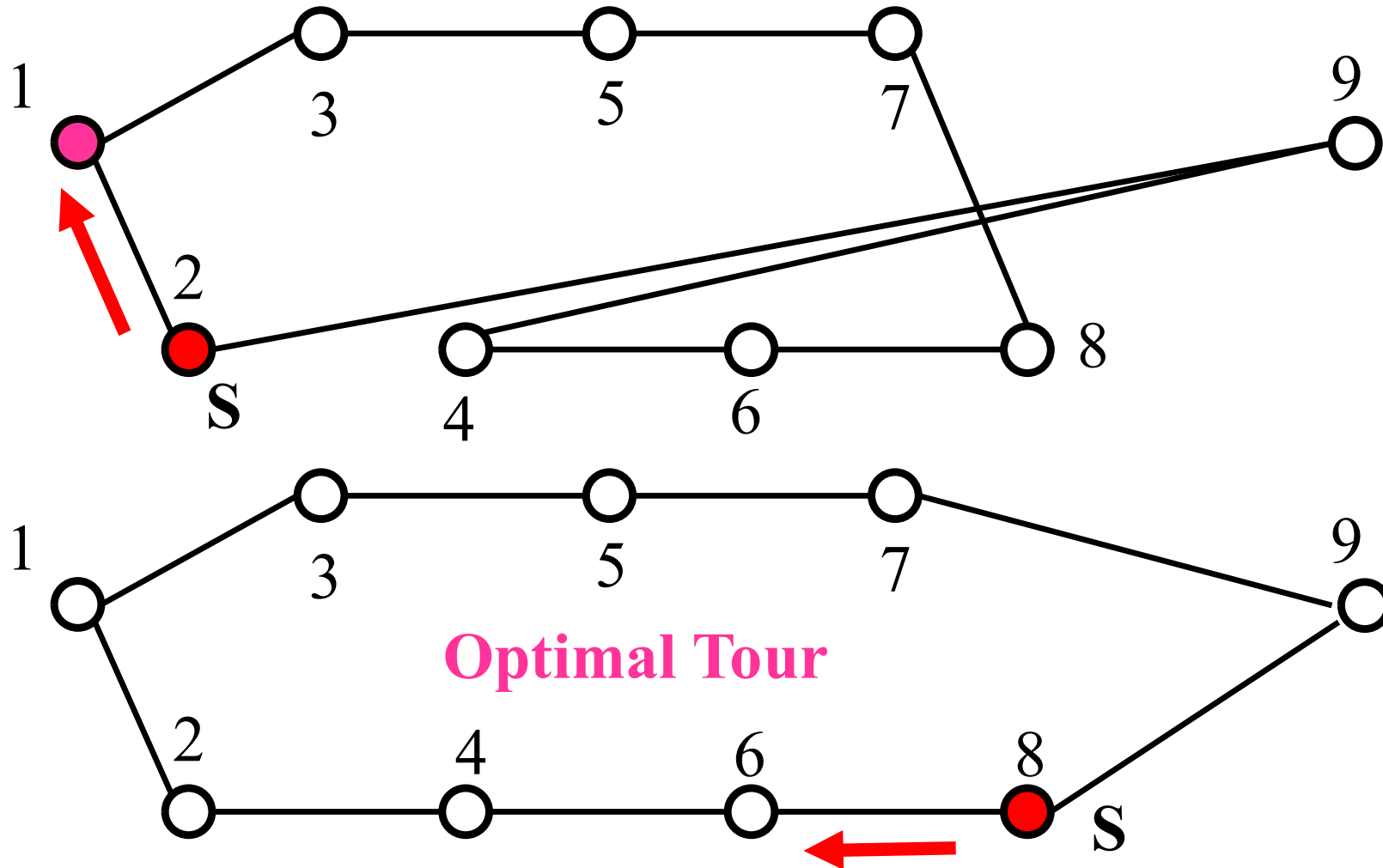
Exact: Impractically slow for large problems



One Simple Idea

To use the greedy algorithm from all cities (n times).

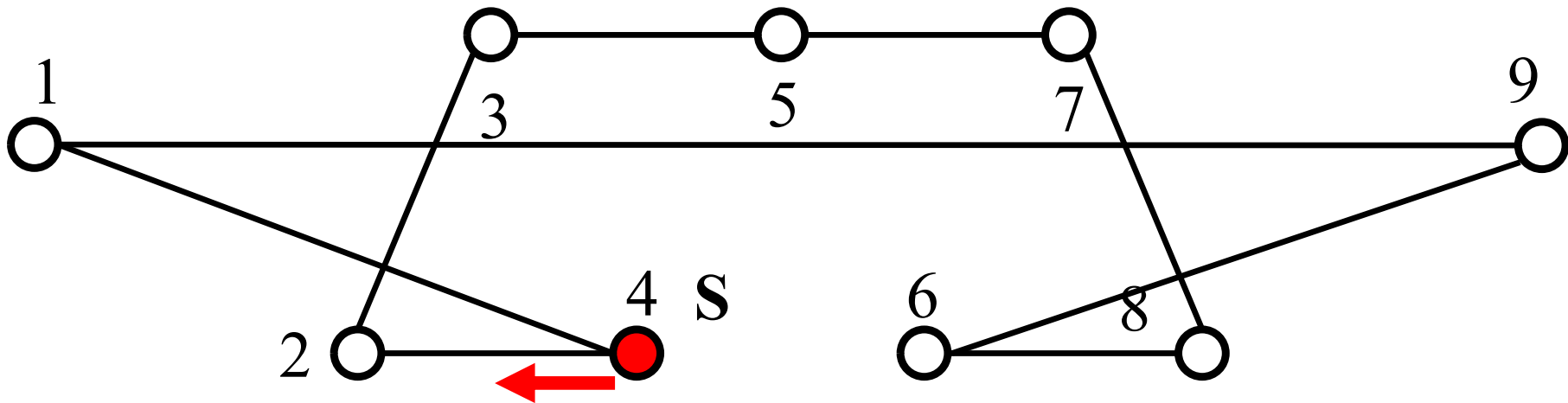
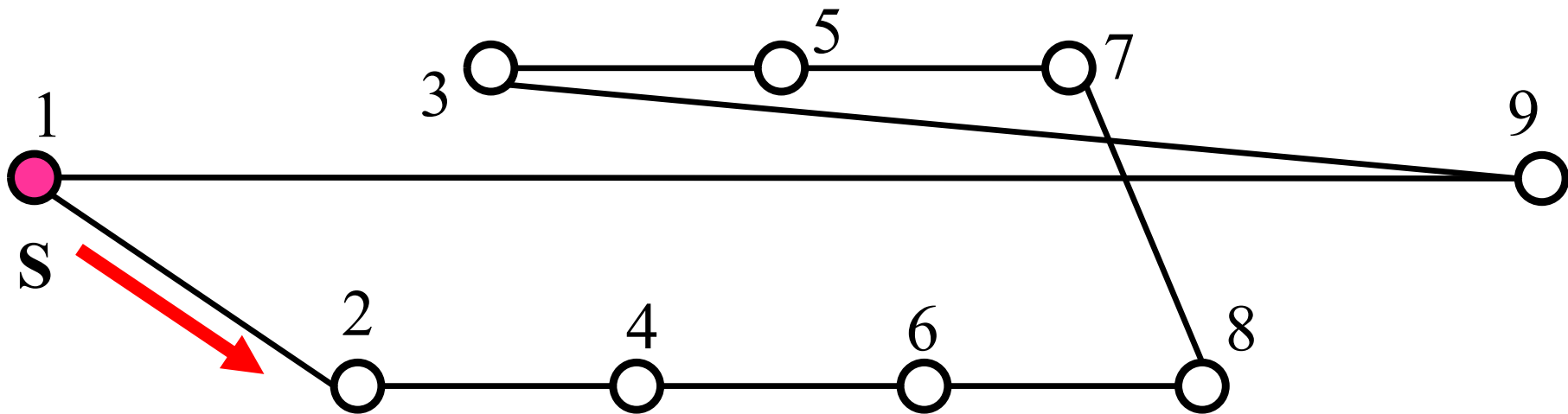
- Significant improvement on some problems



One Simple Idea

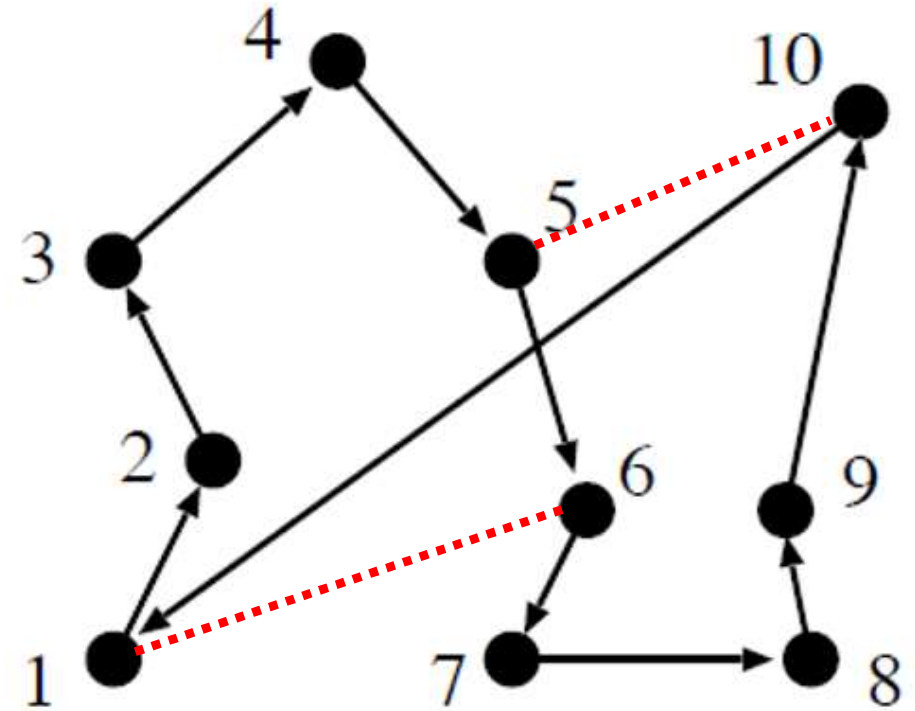
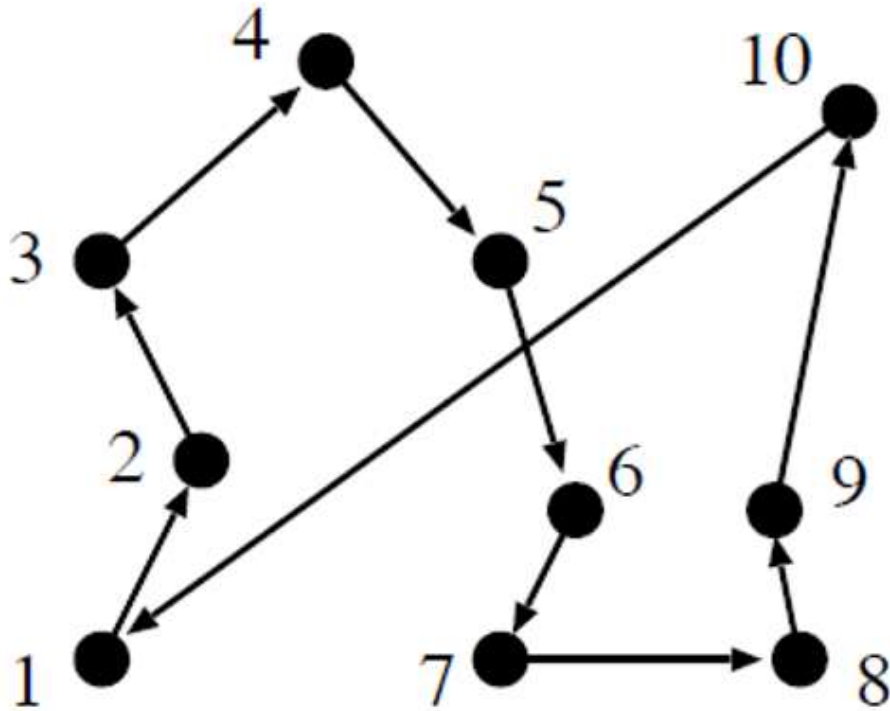
To use the greedy algorithm from all cities (n times).

- Significant improvement on some problems
- Minor (almost no) improvement on other problems



Another Idea

To try to improve the current tour: Local Search (LS)



Basic Mechanism (iterative improvement):

1. Slightly modify the current tour.
2. If the current tour is improved, accept the modification.
If the current tour is not improved, do not change it.
3. Go back to 1 for examining another modification.

Local Search (First Move Version)

1. Generate an initial solution $\mathbf{x}$.
2. Iterate the following steps:
 - (i) Generate a neighbor solution $\mathbf{y}$ of the current solution $\mathbf{x}$.
 - (ii) If $\mathbf{y}$ is better than $\mathbf{x}$, replace the current solution $\mathbf{x}$ with $\mathbf{y}$.
 - (iii) If no better solution exists in the neighborhood of $\mathbf{x}$, terminate the execution of the algorithm.

Local search is simple. However, it has a lot of implementation issues such as

- How to generate an initial solution $\mathbf{x} \implies$ **Greedy Solution**
- How to specify the neighborhood of the current solution $\mathbf{x}$,
- How to specify the order of neighbors to be examined
 $\implies$ **Random Order**

Local Search (Best Move Version)

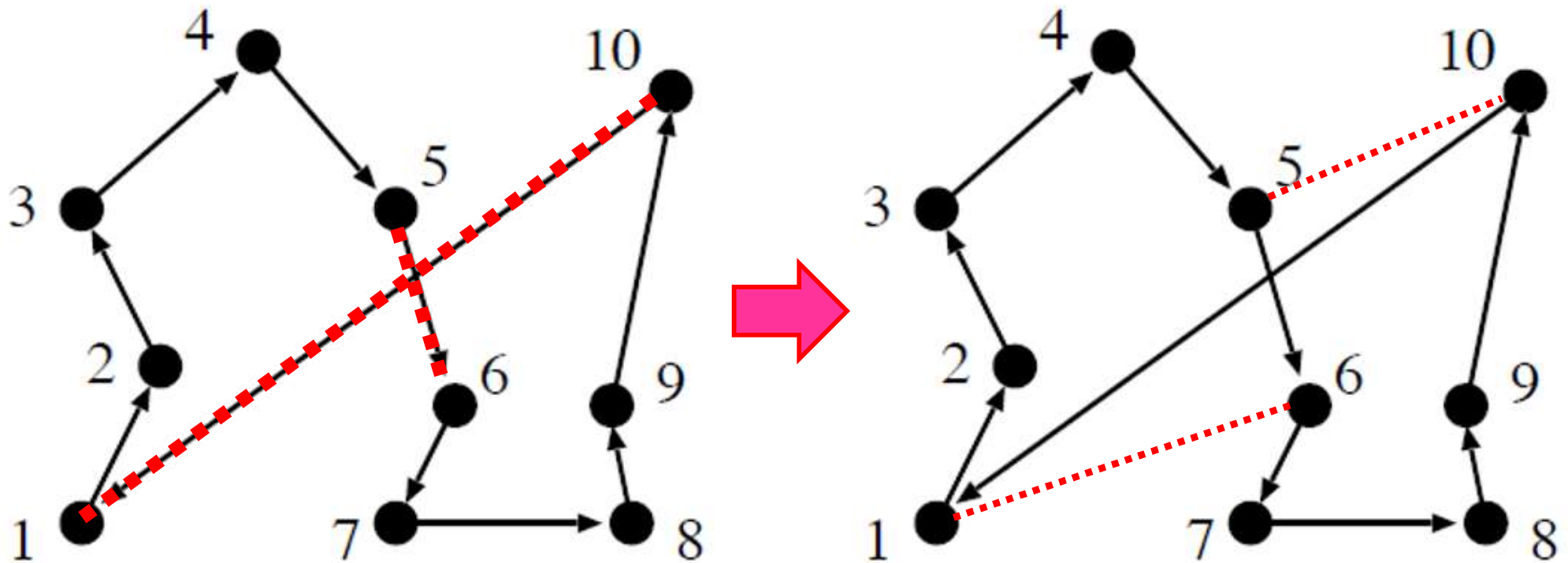
1. Generate an initial solution $\mathbf{x}$.
2. Iterate the following steps:
 - (i) Find the best solution $\mathbf{y}$ in the neighborhood of the current solution $\mathbf{x}$.
 - (ii) If $\mathbf{y}$ is better than $\mathbf{x}$, replace the current solution $\mathbf{x}$ with $\mathbf{y}$. Otherwise terminate the execution of the algorithm.

Local search is simple. However, it has a lot of implementation issues such as

- How to generate an initial solution $\mathbf{x} \implies$ **Greedy Solution**
- How to specify the neighborhood of the current solution $\mathbf{x}$,
- ~~How to specify the order of neighbors to be examined~~

The main issue in the design of Local Search

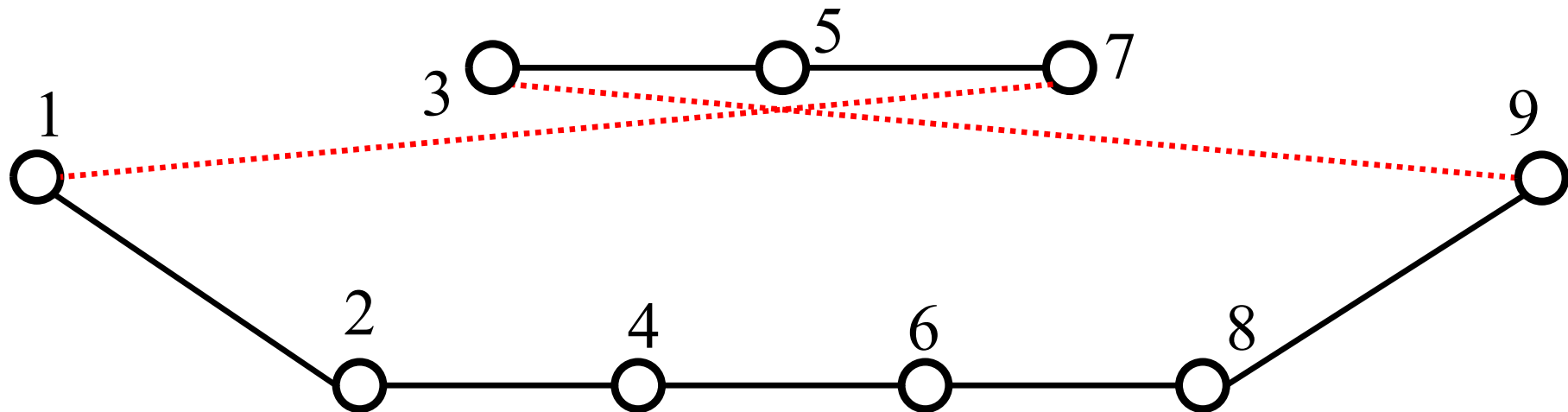
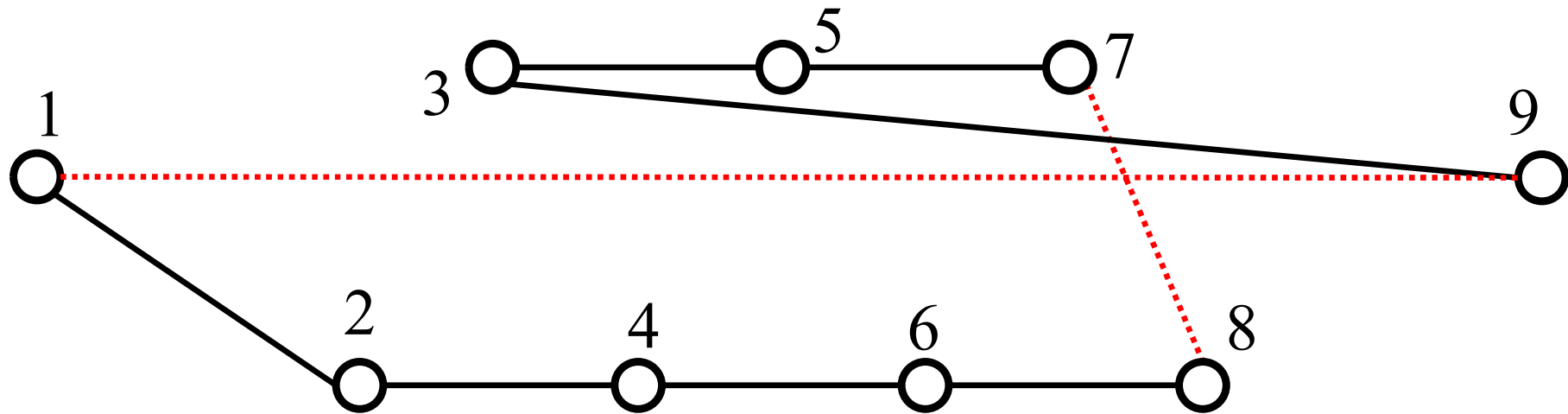
How to specify the neighborhood of the current solution x



Arbitrary Two-Edge Change

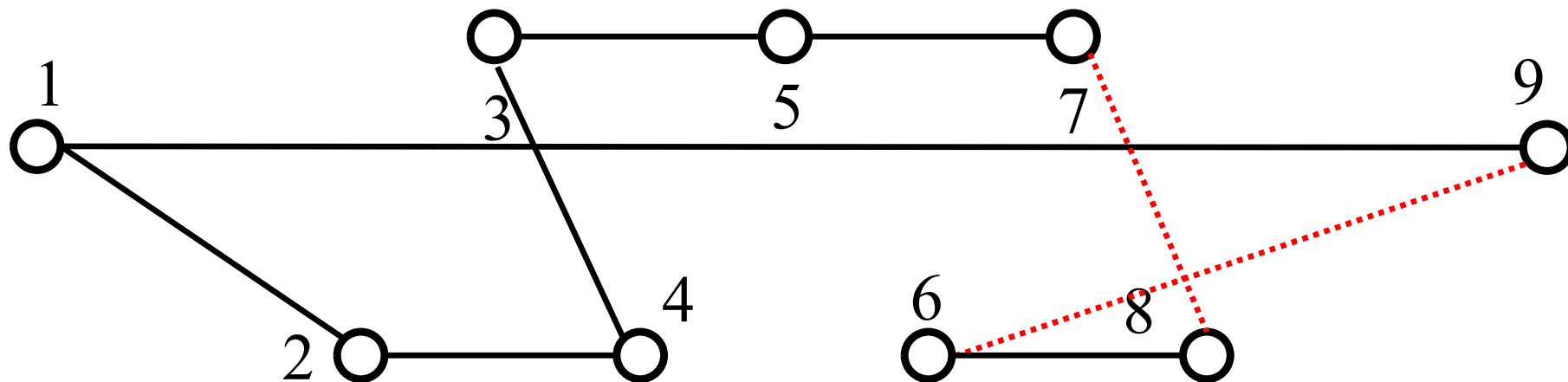
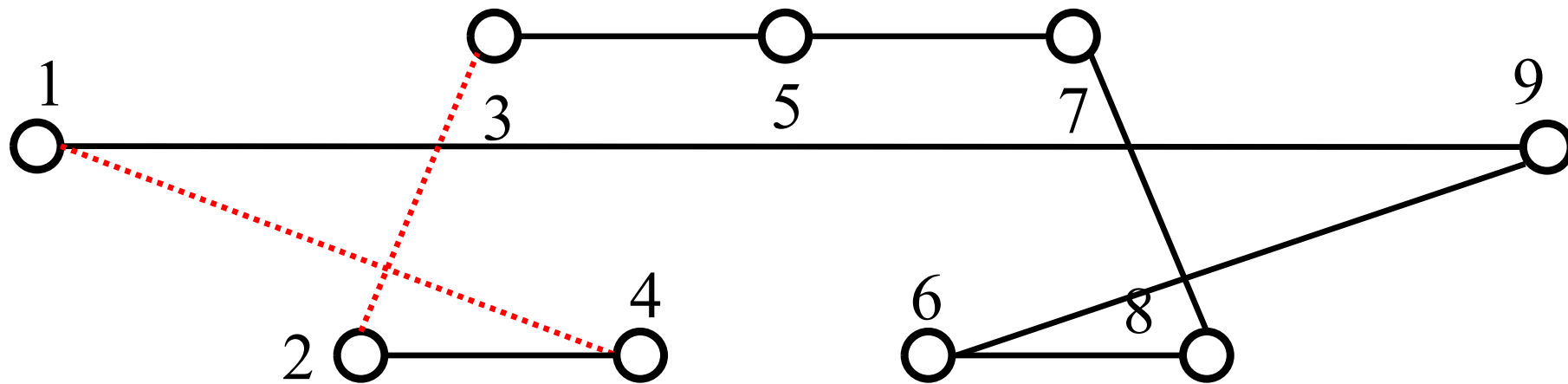
The main issue in the design of Local Search

How to specify the neighborhood of the current solution x



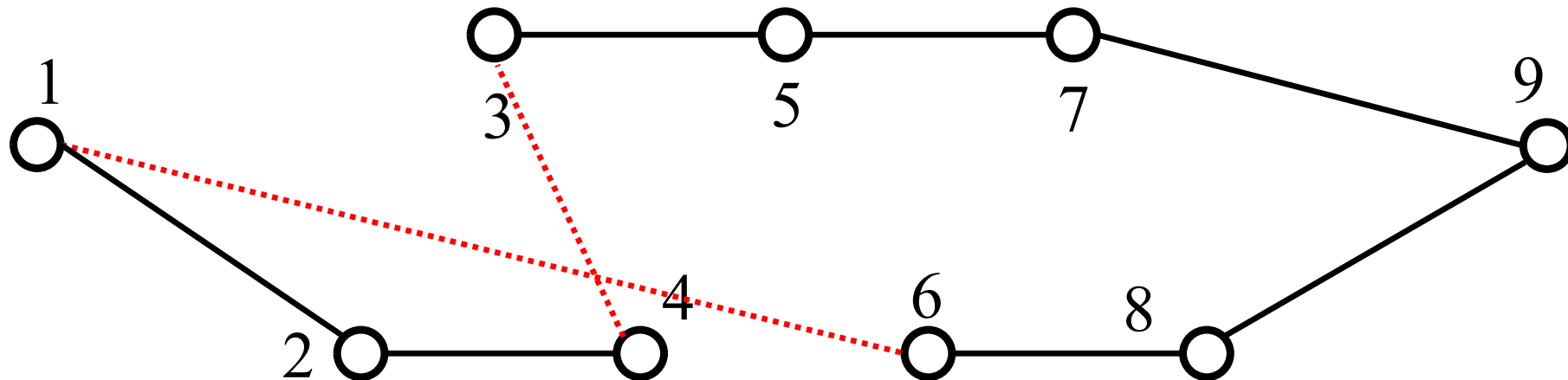
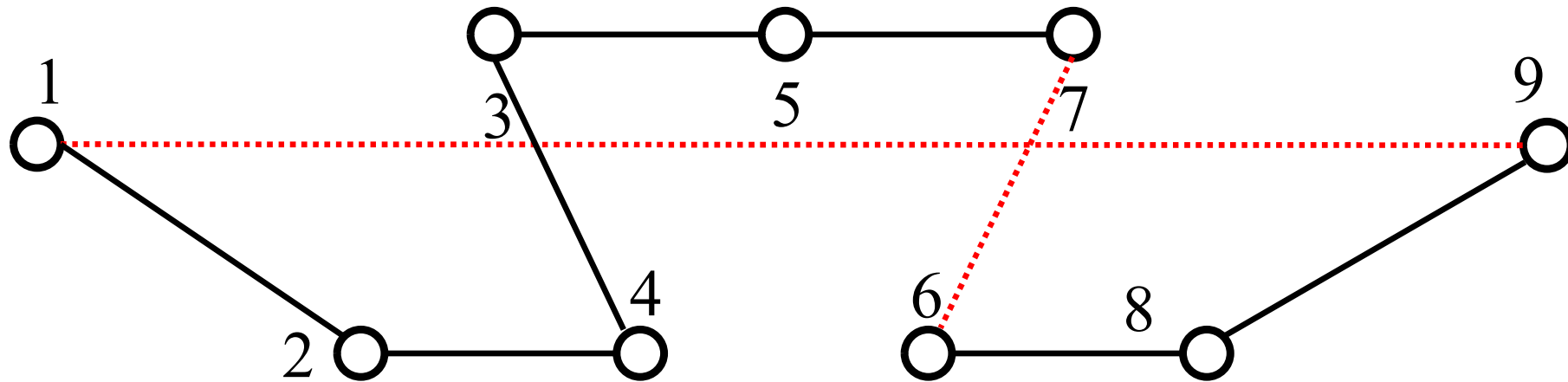
The main issue in the design of Local Search

How to specify the neighborhood of the current solution x



The main issue in the design of Local Search

How to specify the neighborhood of the current solution x

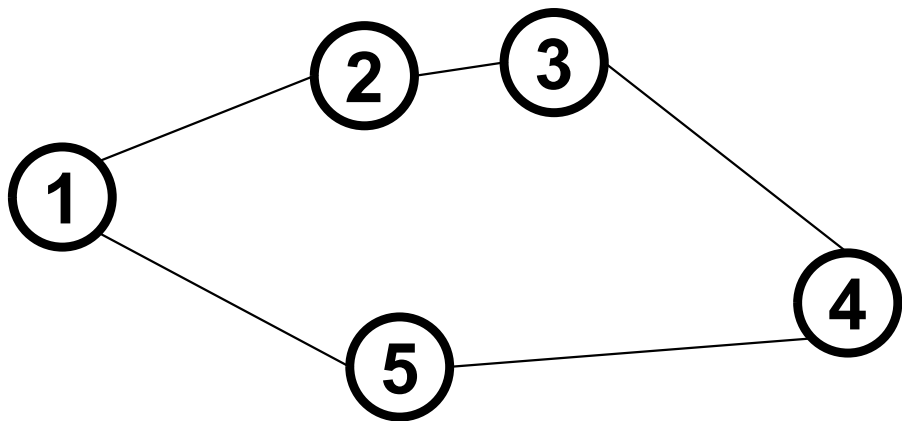


1st Week Lab Session

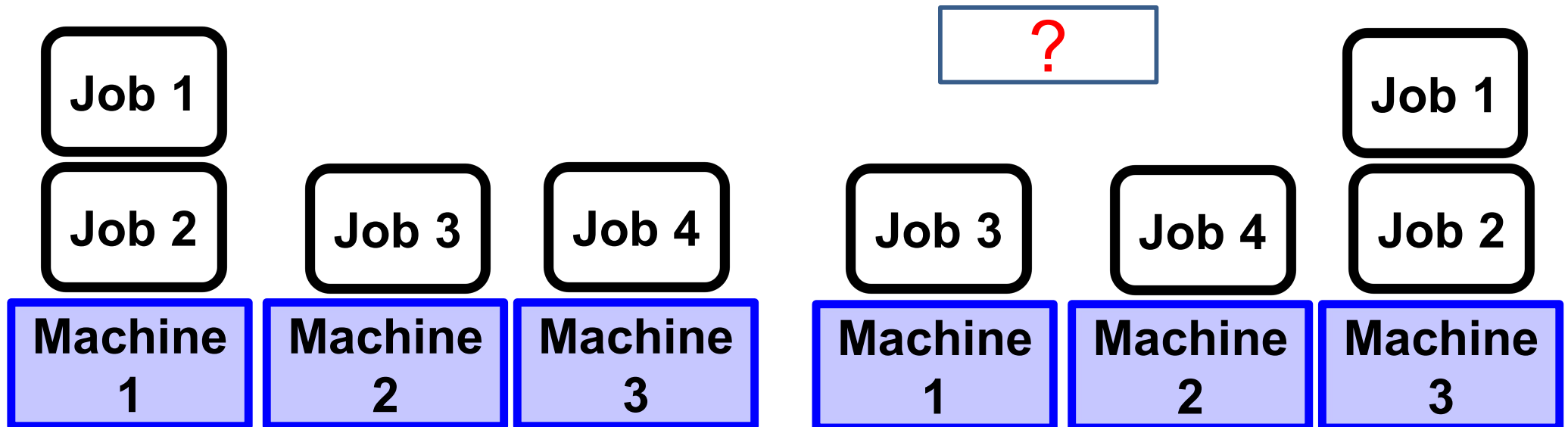
Prepare a PowerPoint file to explain how to solve each question. You will be asked to explain how to solve each question using your file at the end of lab session (e.g., 15:30 - 15:50)

Question 1: An n -city TSP problem (i.e., to find the shortest tour to visit all the given n cities and return to the start city). The problem size is the total number of different tours. This is not $n!$ since many tours can be viewed as the same tour (with respect to the tour length). For example, in the following 5-city TSP problem, the tour 123451 is the same as 154321, 234512, 215432 and some other tours. **How many different tours does an n -city TSP problem have?**

?



Question 2: A 3-machine 4-job load balancing problem. This problem is to find the best assignment of four jobs to three identical machines. Note that some different assignments are viewed as the same solution since all machines are identical. For example, the assignment of all jobs to Machine 1 is viewed as the same solution as the assignment of all jobs to Machine 2 (or Machine 3). The following two assignments are viewed as the same solution. **How many different solutions does the 3-machine 4-job load balancing problem have?**



This problem is the same as a grouping problem of four jobs into three groups: $\{\{J1, J2\}, \{J3\}, \{J4\}\}$ is the same as $\{\{J3\}, \{J4\}, \{J1, J2\}\}$.

Question 3: A 2-machine n -job load balancing problem. As in Example 2, some different assignments (e.g., two assignments below) are viewed as the same solution since the two machines are identical. How many different solutions does the 2-machine n -job load balancing problem have?

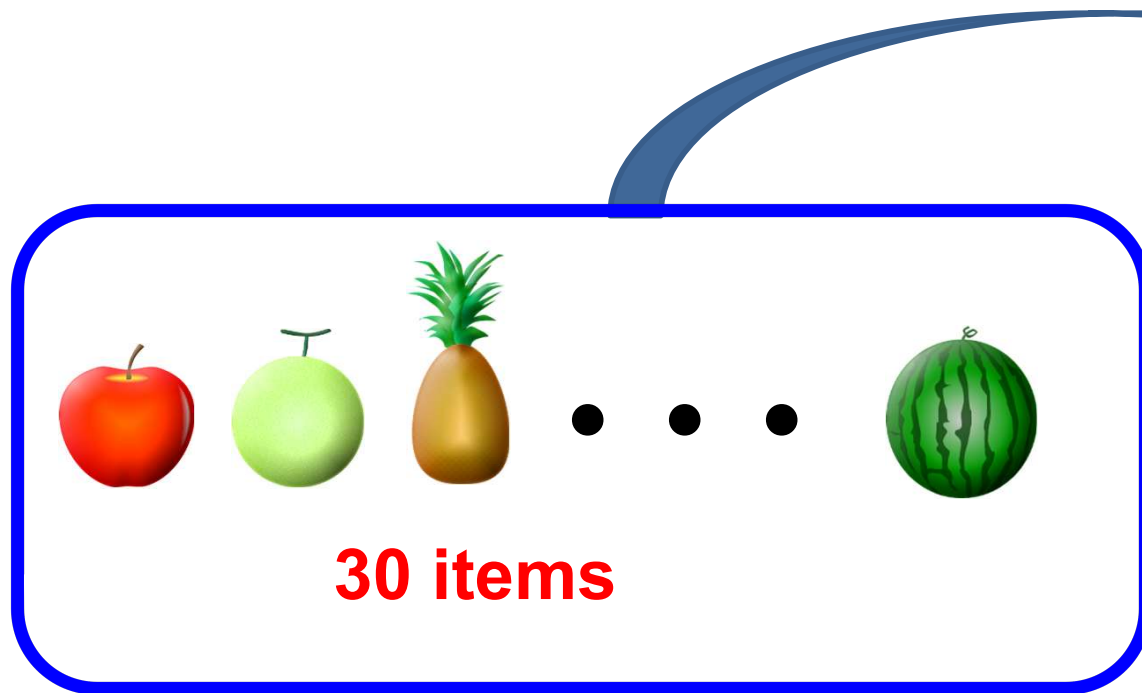
?



A 2-machine n -job load balancing problem is different from subset selection from n jobs. In the above two figures, the selection of {Job 1 and Job 2} is different from the selection of {Job 3} in subset selection (whereas these two solutions are the same in load balancing).

Question 4: A 30-item knapsack problem (to find the best item set of the given 30 items under a capacity constraint). The set of all different solutions includes the selection of no items and the selection of all items. **How many different solutions does the 30-item knapsack problem have?**

?



Capacity: 10 Kg

This problem is the same as subset selection.

Question 5: How many items are needed in an m -item knapsack problem to have a similar search space size to a 1000-city TSP problem.

Question 6: How many cities are needed in an n -city TSP problem to have a similar search space size to a 1000-item knapsack problem.